

PRODUCTION-TARGET SYSTEM ARCHITECTURE

DrishtiGuard

On-device visual perception and privacy firewall for lightweight browser agents

Fail-closed privacy before egress. Local policy before action.

A browser-native architecture that combines DOM and visual evidence, minimizes every outbound modality, verifies the exact transmitted bytes, and treats a remote model response as untrusted intent—not executable authority.

Problem statement	SIH26171 — On-device Visual Perception for Light-weight Browser Agents
Organization	Department of Space / Indian Space Research Organisation (ISRO)
Category / theme	Software / Smart Automation
Architecture status	Baseline v1.0 — production target; hackathon MVP scoped separately
Verified	4 September 2026 against the live official SIH problem-statement page
Primary audience	SIH team, technical judges, security/privacy reviewers and implementation leads

[Architecture](#) • [Threat model](#) • [Privacy model](#) • [Contracts](#) • [Benchmarks](#) • [Roadmap](#) • [ADRs](#)

Document control and reader's guide

IMPORTANT STATUS

This report is a production-target architecture and evidence plan. It does not claim that a production-ready implementation exists. The hackathon deliverable is a measured, narrow vertical slice with fail-closed controls.

Baseline controls

Field	Value
Document ID	DG-ARCH-001
Version	1.0
Status	Approved architecture baseline for implementation planning
Owner	DrishtiGuard architecture lead; sponsor decisions remain in the open-decision register
Classification	Project internal; contains no real personal data or ISRO operational data
Change control	Material trust-boundary, privacy-gate or action-policy changes require a new ADR and benchmark impact assessment
Design preset	Standard business brief with editorial technical cover; Letter portrait, 1-inch margins, fixed-width tables

How to use this baseline

- Start with Part I for the end-to-end decision, system boundaries, components, flows and deployment model.
- Treat Parts II and III as mandatory security and privacy release criteria—not optional quality guidance.
- Generate implementation types from the versioned schema described in Part IV and reject unknown wire fields.
- Use Part V to turn SIH scoring criteria into measurable component outputs and honest claims.
- Use Part VI to protect the hackathon scope; production readiness begins only after the listed pilot and audit gates.
- Record any material deviation in a superseding ADR before code and benchmark baselines diverge.

Table of contents

Part	Contents
I	Production target architecture — decisions, system boundaries, components, flows and deployment
II	Threat model — STRIDE, agent abuse cases, prompt injection, supply chain and residual risk
III	Privacy model — classification, lifecycle, cross-modal sanitization and exact-byte verification
IV	API and contract specification — local capabilities, strict wire schemas and action unions
V	Benchmark and evaluation plan — SIH traceability, datasets, metrics and release gates
VI	Implementation roadmap — 48-hour MVP, post-selection, pilot and production gates
VII	Architecture decision records — ten accepted decisions and their consequences
Handoff	Immediate implementation order and approval boundary

DRISHTIGUARD ARCHITECTURE BASELINE

Part I — Production target architecture

Decision, scope, trust zones, components, flows, deployment and operational gates

Controlled source: docs/ARCHITECTURE.md

Problem statement: SIH26171 - On-device Visual Perception for Light-weight Browser Agents

Organization: Department of Space / Indian Space Research Organisation (ISRO)

Category and theme: Software / Smart Automation

Document status: Architecture baseline v1.0

Verified: 4 September 2026 against the live official SIH problem-statement page [R1]

Interpretation: This is a production-target design and a hackathon implementation plan. It is not evidence that a production-ready system exists.

1. Executive decision

Build **DrishtiGuard** as a browser-native, fail-closed pixel firewall for AI browser agents. The browser extension performs capture, semantic extraction, local perception, privacy compilation, final-byte leakage verification, action validation, user confirmation and execution. A remote vision-language model receives only a verified, minimized representation and returns exactly one schema-constrained action over an opaque element ID.

The central design decision is not merely to detect PII and cover pixels. DrishtiGuard makes privacy and execution safety architectural capabilities:

- The privacy path combines DOM-derived semantics and visual evidence. Privacy masking uses the union of findings; action grounding requires stronger concordance and a live DOM binding.
- A coverage analyzer distinguishes "nothing detected" from "the entire visible surface was examined." Unknown or unprocessed regions are masked or block egress.
- The privacy compiler sanitizes every outbound modality: image, scene graph, accessible names, attributes, URLs, user task and action arguments.
- A separate verifier examines the exact encoded image and serialized request. It authorizes only that payload digest.
- Only the outbound privacy gateway can send page-derived content. Its input type cannot represent a raw screenshot, raw DOM, token map, selector or user secret.
- Typed placeholders are backed by unguessable, task-scoped token capabilities held in memory locally. Literal text such as `[PAN_1]` is display-only and never resolves by itself.
- The remote model cannot execute code. It returns one validated action; the local policy engine determines risk from effect and context, not merely the action verb.
- Every action is bound to a current snapshot, navigation epoch, mutation epoch and target fingerprint. High-risk actions require extension-owned confirmation and a second revalidation after confirmation.

The strongest honest guarantee is:

Original capture buffers and reversible token mappings are structurally prohibited from egress. Content discovered or classified as sensitive is blocked or transformed, with residual detection risk measured on a declared benchmark.

No architecture that uses probabilistic PII recognition can truthfully guarantee that every unknown sensitive fact will always be detected. Universal "100% privacy" is therefore not a permitted product claim.

2. Problem verification and architecture drivers

The official SIH page identifies SIH26171, the stated title, ISRO, Software, Smart Automation, Chrome and Firefox, local vision processing, a privacy-preserving filter, a centralized LLM/VLM and an end-to-end assisted task [R1]. It assigns the following evaluation weights:

Evaluation metric	Weight	Architectural driver
Visual-context accuracy	25%	DOM/vision fusion, coordinate integrity, live element binding
PII precision and recall	20%	Ensemble recognition, per-class calibration, coverage analysis
Redaction precision	20%	Privacy compiler, fresh bitmap generation, exact-payload verifier
Client resource utilization	20%	Quantized tiered runtime, ROI OCR, buffer reuse, backpressure
End-to-end latency	15%	Event-driven capture, cache, cancellation, one-step plans

Privacy is a release gate even though the competition uses a weighted score. A faster pipeline is not allowed to omit a detector or verifier.

2.1 Prior art and defensible differentiation

GUIGuard already defines recognition, protection and protected execution for GUI agents, and reports that privacy recognition remains a bottleneck [R10]. Newer work also studies type-preserving placeholders [R11] and context-aware exposure control [R12]. DrishtiGuard must not claim that redaction or typed anonymization is unprecedented.

Its defensible engineering advance is the combination of:

1. browser-native dual-signal collection with explicit coverage accounting;
2. scene-graph-first disclosure minimization;
3. final-byte verification bound cryptographically to the outgoing request;
4. task-random typed token capabilities rather than model-resolvable strings;
5. a sole, schema-limited egress path;
6. opaque target binding, stale-plan rejection and post-confirmation revalidation;

7. capability-driven Chrome/Firefox adapters; and
8. benchmark evidence that jointly reports privacy, utility and resource cost.

These are system differentiators, not a research-novelty claim until measured against current baselines.

3. Scope

3.1 In scope

- A WebExtensions MV3 browser extension built from one TypeScript codebase, with Chromium and Firefox adapters.
- Explicit user activation using temporary `activeTab` authority where supported [R3, R5].
- Visible-viewport capture and DOM-derived accessibility semantics for the top-level document, accessible same-origin frames and open shadow roots.
- Local visual detection, region-of-interest OCR, PII classification, typed tokenization, masking and final leakage verification.
- A minimized remote request containing a sanitized task, sanitized element graph and, only when necessary, a newly encoded sanitized image.
- A stateless remote planning endpoint using an open/open-weight VLM, with Qwen3-VL-4B-Instruct as a benchmark candidate rather than a preselected winner [R13].
- One action per model response, local validation, deterministic policy, user confirmation for consequential effects and post-action verification.
- Privacy-safe metrics, fault injection, contract tests and a synthetic employee travel-claim demonstration.

3.2 Non-goals for the hackathon MVP

- Universal automation across the open web.
- Native-trusted clicks inside arbitrary canvas, browser chrome, protected PDF viewers, closed shadow DOM or inaccessible cross-origin frames.
- Login, passwords, OTP entry, payment, file upload, deletion or permission changes.
- Full native browser accessibility-tree access. A minimally privileged WebExtension observes DOM/ARIA-derived semantics, not the complete platform accessibility tree.
- Guaranteed detection of every possible confidential fact, steganographic signal or inferential disclosure.
- Full offline planning. Privacy processing remains local when offline, but remote planning is unavailable.
- Safari support, enterprise fleet management, production data residency certification or a general-purpose agent marketplace.

3.3 Prototype vertical slice

The end-to-end slice is a synthetic employee travel-claim portal containing a synthetic name, face, employee ID, PAN-shaped value, Aadhaar-shaped value, account/IFSC pair, email, invoice image, claim amount and submit control. The user task is: "Check whether the claim is within the declared policy limit and submit after confirmation."

The local system computes the privacy-safe derived fact `within_policy_limit=true`; it does not need to disclose the exact amount if financial amounts are classified as sensitive. The hard surface is an invoice rendered into canvas or an image with missing ARIA metadata. All values are deterministically generated synthetic values that are invalid against real registries.

4. Recommended assumptions

These defaults unblock design. They must be confirmed before implementation commitments.

Decision area	Recommended working assumption
Team and time	Six members with pre-event preparation and a 36-48 hour finale sprint
Primary client	Modern Chromium on Windows x64; test the latest two stable versions at benchmark freeze
Compatibility client	Current Firefox stable using WASM; functional support first, measured parity later
Client hardware	4+ CPU cores, 16 GB RAM, WebGPU optional; integrated GPU is the baseline case
Page scope	Visible viewport on explicitly supported HTTP(S) origins
Action scope	Top-level and accessible same-origin DOM-backed targets only
Concurrency	One active task per tab; one active inference job per task
Remote service	Self-hosted modular monolith on an India-region or approved on-prem GPU host
Authentication	Short-lived demo token for MVP; OIDC Authorization Code + PKCE for pilot, no embedded client secret
PII classes	Credentials, OTPs, tokens, Aadhaar, PAN, account/IFSC, cards, employee IDs, names, faces, emails, phones, addresses, medical and financial data
Data retention	Raw frames and token maps in memory only; sanitized request bodies have zero server retention by default
Telemetry	Content-free metrics only; opt-in diagnostics; no prompt, response, OCR or screenshot logging
Data residency	India-region processing pending written ISRO direction
Offline behavior	Local capture may be analyzed, but no remote plan and no privacy downgrade
High-risk action	Any submit/write/share/delete/upload/download/auth/payment, cross-origin navigation or ambiguous click
Source publication	Secret-free builds; publish only after dependency, model-license and security review
Expected volume	Unknown; start with bounded stateless replicas and one in-flight request per task

Numerical performance and accuracy values in this document are targets until the hardware, browser, model hashes, dataset versions and sample sizes are frozen.

5. System context

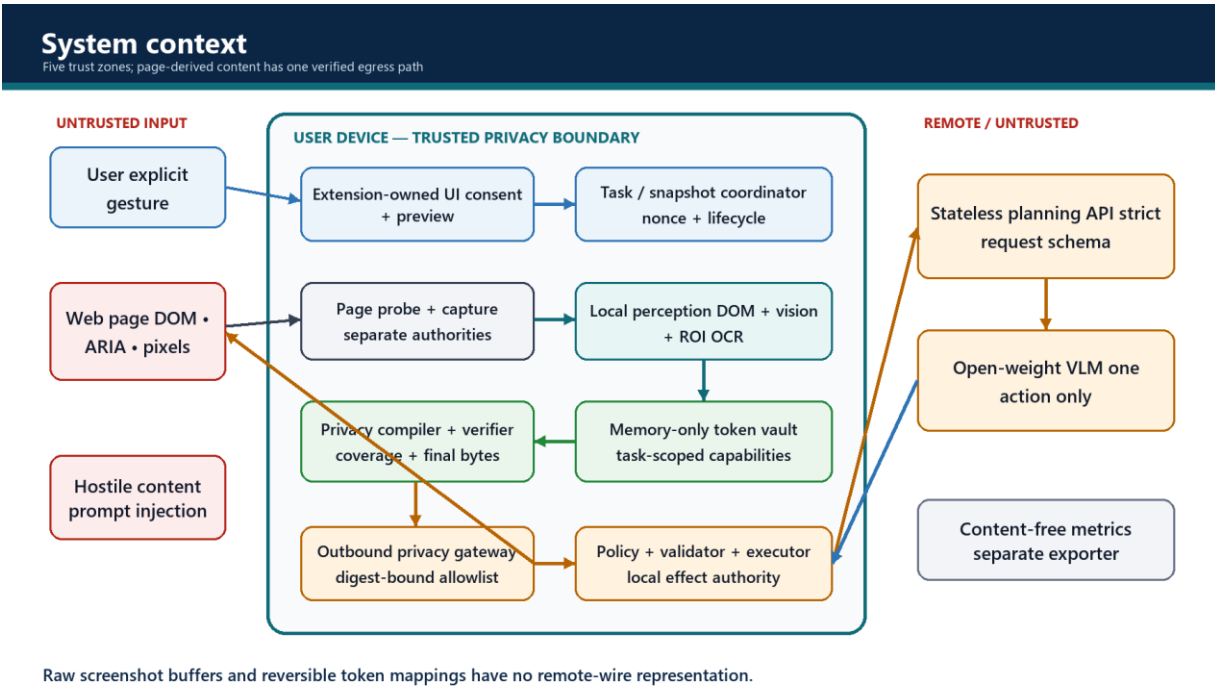


Figure 1. System context and sole page-data egress path.

The trusted computing base includes the signed extension package, bundled model runtime and weights, local browser extension APIs, the privacy-policy configuration, contract validators, remote service image and release pipeline. The webpage, remote model output, network, third-party content and telemetry backend are not trusted with raw page data.

6. Invariants and enforcement

Invariant	Enforcement point	Proof or test
Raw capture bytes cannot cross egress	Gateway accepts only <code>VerifiedEnvelope</code> ; raw handle type is non-serializable	Static import rule, contract negative test, intercepted-request canary scan
Token mappings remain device-local	Vault is in an active local compute host; no storage serializer	Schema rejects mapping keys; storage and network scans
Every page-derived modality is sanitized	Privacy compiler handles image, graph, task, URLs and arguments	Cross-modal seeded canary suite
Critical uncertainty never silently passes	Coverage gate, union masking, bounded retry then block	Fault injection and low-confidence cases
Critical masks are irreversible	Solid opaque fill or semantic replacement in a fresh bitmap	Alpha/metadata inspection and adversarial OCR
Page content cannot change policy	System/task/observation channels are separated; policy is local	Prompt-injection suite

Invariant	Enforcement point	Proof or test
Remote output cannot execute code	Closed action union; no JS, selectors, coordinates or arbitrary URLs	JSON Schema fuzzing
Every action uses an opaque target	Remote schema requires task-random element ID	Invalid-plan rejection tests
Stale actions cannot execute	Snapshot, document, navigation and mutation checks before action	Race and replay tests
Consequential actions require confirmation	Effect-based policy and digest-bound extension UI	Confirmation bypass tests
Telemetry carries no content	Dedicated safe event schema and drop-on-scrub-failure	Sink inspection and canary scan
Permissions remain minimal	<code>activeTab</code> , <code>scripting</code> , scoped endpoint origin; optional host grants	Manifest audit per browser build
Failures do not weaken privacy	Certified fallback matrix; otherwise block	Dependency/model/runtime fault injection
Contracts are versioned	Draft 2020-12 JSON Schema and generated TypeScript	Compatibility and consumer-driven contract tests
Tests use no real personal or ISRO data	Deterministic synthetic generator and corpus review	Dataset manifest audit
User task is sensitive until sanitized	Task sanitizer runs before remote request construction	Seeded task-string leak tests
Only the exact verified payload may leave	Verifier signs digest; gateway recomputes immediately before send	Mutation-between-verify-and-send test

The "single gateway" prevents accidental leakage by trusted code; it does not defeat a fully compromised extension. Signed releases, pinned dependencies, reproducible builds and review are therefore part of the privacy boundary.

7. Browser-extension container architecture

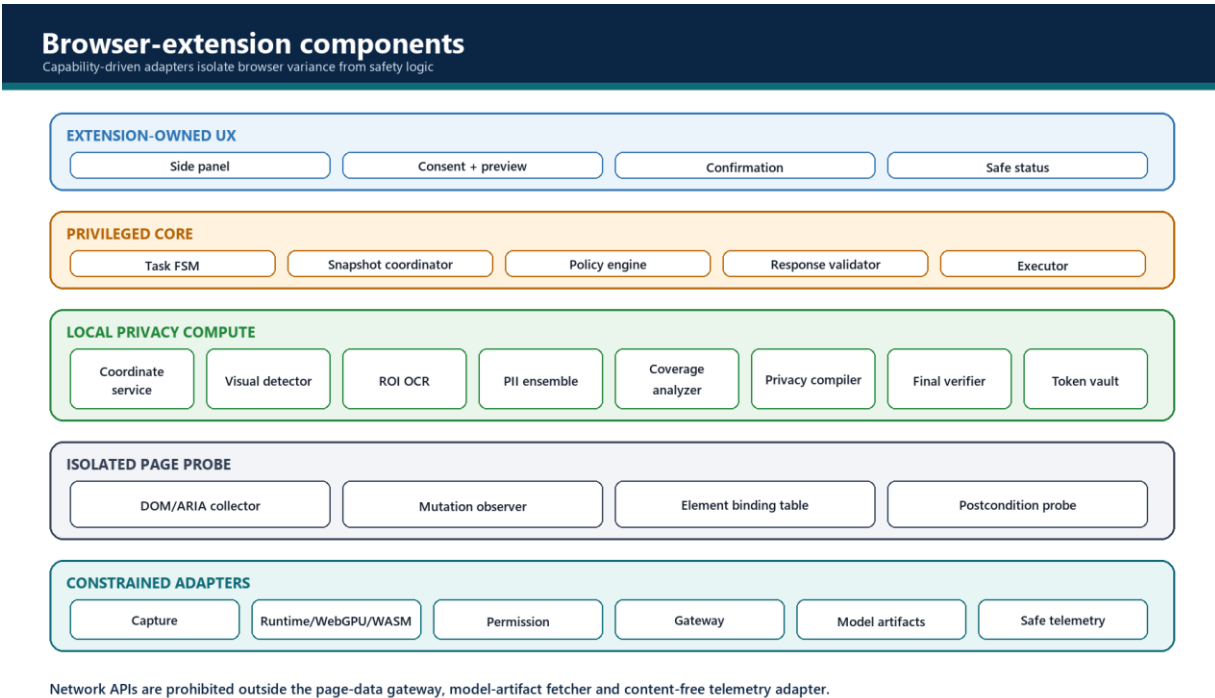


Figure 2. Browser extension component boundaries and constrained adapters.

7.1 Extension shell and compute lifecycle

Use TypeScript, WXT or an equivalent framework that emits browser-specific manifests from one codebase [R6]. Browser-name checks are permitted only inside adapters. The extension-owned side panel is the user-visible task and confirmation surface. A short-lived MV3 service worker coordinates browser events; it must not be the sole owner of an in-memory token vault because it can be suspended.

The active compute host owns the task vault and dedicated workers. Chromium may use an offscreen document or a side-panel/extension page where supported; Firefox uses its supported extension-page/worker lifecycle. If the host disappears, all handles become invalid and the task moves to `CANCELLED_VAULT_LOST`. It never recreates mappings from logs or persistent storage.

Privileged messages include schema version, task ID, random session nonce, sender role and monotonic sequence. Every receiver checks `runtime.id`, sender tab/frame, task membership and replay window. Content scripts are treated as compromised-input adapters, not peers of the gateway.

7.2 Permissions and CSP

The default manifest requests `activeTab` and `scripting`, with a narrowly scoped host permission for the configured reasoning endpoint. Chrome documents `activeTab` as temporary access following an explicit user gesture [R3]. Firefox support for `captureVisibleTab` with `activeTab` depends on supported versions; Firefox 125 and earlier required `<all_urls>`, so the minimum version must be frozen in the compatibility matrix [R5].

The extension CSP permits only packaged scripts and objects. `wasm-unsafe-eval` is allowed only in extension contexts that require packaged ONNX Runtime Web WASM; ordinary `unsafe-eval`, inline scripts and remote code are forbidden [R4]. No API secret or reusable private key is bundled.

7.3 Capture coordinator

The coordinator is event-driven because Chrome limits `captureVisibleTab` to two calls per second and documents it as expensive [R2]. It debounces mutations, enforces one in-flight capture per tab and cancels superseded work.

A coordinated snapshot uses a read-capture-read protocol:

9. read `document_id` where available, `navigation_id`, viewport, scroll, zoom and `mutation_epoch`;
10. collect DOM-derived semantics and capture visible pixels;
11. read the same state again;
12. discard the pair if any identity or layout-critical value changed.

This is not truly atomic, but it makes inconsistency detectable. Dynamic video or canvas invalidates changed-region reuse unless its pixels are covered by a full-frame diff.

7.4 Coordinate-normalization service

The canonical coordinate space is root visual-viewport CSS pixels. Every region carries `snapshot_id`, `document_frame_id`, coordinate-space ID, viewport dimensions, screenshot dimensions, observed scale, scroll, timestamp, source and confidence.

The screenshot transform is derived from actual captured dimensions:

```
scale_x = encoded_bitmap_width / visual_viewport_width_css  
scale_y = encoded_bitmap_height / visual_viewport_height_css
```

Recorded DPR and browser zoom are diagnostics, not the sole transform. Same-origin frame rectangles are accumulated into the root coordinate system. Affine transforms are represented by matrices; perspective transforms, out-of-bounds boxes or scale mismatch beyond tolerance block the snapshot. Cross-origin frames remain opaque visual regions unless separately permitted and instrumented.

7.5 DOM and ARIA collector

The collector produces a versioned local element graph with task-random opaque IDs, parent ID, tag class, role, accessible-name candidates, label relationships, input metadata, state, bounding rectangle, visibility, frame identity and evidence source. It traverses open shadow roots and accessible same-origin frames.

It does not transmit raw `innerHTML`, arbitrary attributes, full URLs, values or selectors. `href`, `src`, `title`, `alt`, `placeholder`, `autocomplete`, data attributes and form values are sensitive until compiled. Password, OTP and authentication-token fields are represented only as `PRESENT_AND_FORBIDDEN`, never read as task input.

7.6 Visual perception, OCR and coverage

The model runtime is isolated from the UI thread in dedicated workers. ONNX Runtime Web provides WASM broadly and WebGPU on supported Chromium configurations; the official support matrix does not show equivalent Firefox WebGPU support, making WASM the planned Firefox path [R7].

The runtime uses signed, hashed, quantized model tiers:

- **Tier A:** WebGPU detector/recognizer with static-shape optimization on validated Chromium hardware.
- **Tier B:** WASM SIMD detector/recognizer with sequential execution and buffer reuse.
- **Tier C:** conservative DOM-only local operation on strictly supported pages; no screenshot egress and no automation of opaque visual regions. This is a different capability mode, not a silent privacy downgrade.

OCR runs on detected text-bearing regions and suspicious image/document regions, batches crops and caches results by salted local region hash plus snapshot lineage. PaddleOCR 3.5 includes an official PaddleOCR.js package built around ONNX Runtime Web and OpenCV.js, with worker support and PP-OCRV5 mobile detection/recognition assets [R8, R22]. It is the leading spike candidate, not an assumed production choice: MV3 asset packaging, CSP and cross-origin-isolation behavior, Firefox operator coverage, browser-measured latency, license inventory and compressed/resident size still require validation. The architecture depends on an [OcrEngine](#) interface rather than one implementation.

The coverage map divides the viewport into tiles/regions and records `SCANNED`, `MASKED`, `SAFE_BY_POLICY` or `UNKNOWN`. Unknown visible regions block the image path or are fully masked. A detector returning no boxes is not equivalent to safe coverage.

7.7 PII-detection ensemble

The ensemble combines:

- strict patterns and validation where available;
- local NER and name/address context;
- input types, autocomplete attributes and nearby labels;
- OCR text and visual face/document/QR/barcode detectors;
- enterprise/site policies and user-declared fields;
- cross-signal agreement and negative-context rules.

PAN uses its official five-letter/four-digit/final-letter structure, plus context; it has no assumed checksum [R19]. Aadhaar-shaped values use format and checksum validation, but are fully masked for DrishtiGuard rather than exposing a last-four representation. UIDAI's masked-Aadhaar convention is useful as domain context, not as this system's egress policy [R18]. IFSC uses the RBI-described four letters, zero, six-character branch structure [R20]. Employee and account identifiers rely heavily on labels and site policy because global patterns are ambiguous.

Presidio is relevant prior art for combining NER, regex, rules and checksums, and its own documentation warns that automated detection cannot guarantee finding all sensitive information [R9]. This motivates an ensemble plus a coverage gate, not a single recognizer.

7.8 Fusion engine and privacy compiler

The privacy canonical set is the geometric union of DOM, OCR, visual and policy findings. Critical boxes receive configurable padding, line-height expansion and overlap merging. If the sources disagree,

privacy uses the larger region. Action grounding uses the opposite posture: an automated action requires a live DOM-backed target plus sufficient semantic/visual concordance.

The compiler assigns a display class and a task-random opaque token ID, for example `PERSON<tkn_7Y...>`. Human-facing previews may show `[PERSON_1]`, but resolution requires the opaque capability. Token entries are bound to task, origin scope, allowed purpose, allowed target field, creation navigation epoch, expiry and use count.

It then:

13. sanitizes task text, element labels, values, attributes and derived arguments;
14. creates padded redaction regions;
15. renders solid opaque masks and optional non-sensitive typed labels into a fresh RGBA-free bitmap;
16. strips metadata and encodes PNG or WebP with fixed safe settings;
17. builds a content-free privacy manifest;
18. hands immutable encoded bytes and JSON to the verifier.

Blur and weak pixelation are prohibited for critical classes.

7.9 Final-artifact leakage verifier

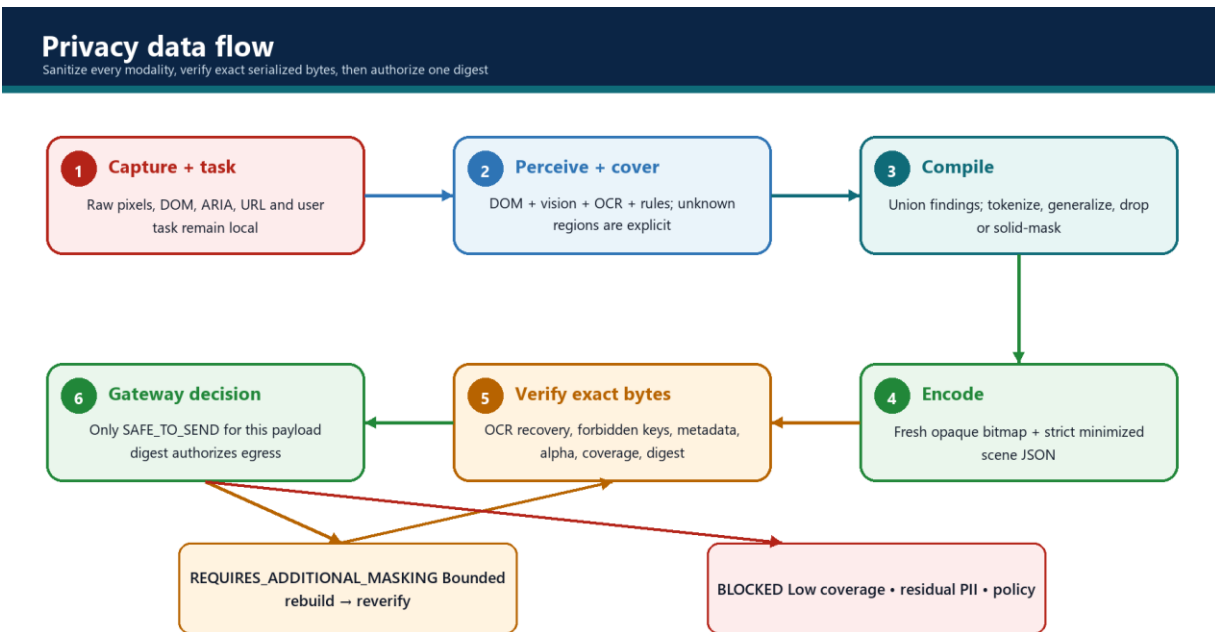


Figure 3. Coverage-aware, exact-byte privacy compilation and egress decision.

The verifier works on the exact final bytes, not an intermediate canvas. It checks:

- known-source values and seeded canaries do not occur in serialized text or decoded image OCR;
- critical patterns/checksums do not survive;
- no forbidden keys, raw handles, DOM HTML, selectors, paths, queries, fragments or mapping objects exist;
- image dimensions, color mode, alpha behavior and metadata match the safe encoder profile;

- every region has a terminal coverage classification;
- mask pixels are opaque and cover the required padded boxes;
- payload size and policy version are allowed.

The preferred verifier uses a different OCR configuration or model from the first pass where feasible. It emits one of `SAFE_TO_SEND`, `REQUIRES_ADDITIONAL_MASKING`, `BLOCKED_LOW_COVERAGE`, `BLOCKED_RESIDUAL_PII` or `BLOCKED_POLICY`. Only `SAFE_TO_SEND` can mint a `VerifiedEnvelope`. Additional masking is retried at most twice before blocking.

The envelope binds `task_id`, `snapshot_id`, policy version, expiry and SHA-256 digest of the exact multipart body. The gateway recomputes the digest immediately before network I/O.

7.10 Outbound privacy gateway

The gateway is the sole page-derived network authority. Build tooling rejects `fetch`, `XMLHttpRequest`, `WebSocket` and remote URL imports outside dedicated network adapters. The extension CSP `connect-src` and host permissions allow only configured origins. A separate model-artifact fetcher and telemetry exporter may exist, but their schemas cannot accept page-derived fields.

The gateway requires current task consent, allowed destination, `VerifiedEnvelope`, matching digest, unexpired snapshot, size limit and supported policy version. It prefers a scene-graph-only request. A sanitized image part is attached only when `visual_context_required=true` and the final verifier approved it.

7.11 Remote reasoning service

The initial server is a modular monolith with HTTP ingress, authentication/rate limit, request validation, prompt construction, VLM adapter, constrained decoding, response validation and content-free metrics. It has no browser-control tool, arbitrary URL fetcher, shell or extension secret.

The model input has three strongly labeled channels:

- immutable system policy and allowed action schema;
- locally sanitized user task; and
- untrusted webpage observation.

Qwen3-VL-4B-Instruct is Apache-2.0 and has documented Transformers/vLLM usage, so it is a reasonable candidate [R13]. It remains replaceable behind `PlannerAdapter`; final selection depends on measured grounding, latency and GPU memory. The server returns exactly one `ActionPlan`. Multi-action plans, JavaScript, CSS/XPath selectors, screen coordinates, raw URLs and unknown keys are rejected.

Ingress, reverse proxy, APM, exception handler and GPU serving logs must all have request-body capture disabled. "Stateless" alone is not evidence of zero retention.

7.12 Action policy, validator and executor

Risk is based on likely effect, target semantics, origin, form context and arguments. A click on "Delete" or "Pay" is high risk even though `CLICK` is syntactically simple.

The validator checks schema version, response nonce, task ID, snapshot ID, navigation ID, mutation epoch, element binding, role, visibility, enabled state, position tolerance and policy. A target fingerprint

includes document frame, role, stable DOM locator held locally, sanitized accessible-name digest, bounding box and ancestor/form context.

High-risk actions are shown in extension-owned UI with sanitized description, origin alias, effect and target. Confirmation is bound to a digest of the exact action and current state, expires after 30 seconds and is single-use. The validator repeats all freshness and target checks after confirmation and immediately before execution.

The MVP executor operates only on accessible DOM-backed elements. It sets values through the page-appropriate event sequence and invokes target actions within the allowed task boundary. Canvas-only and inaccessible cross-origin actions are user-assisted; visual coordinates never become an automated trusted click in the MVP.

8. End-to-end flow and state

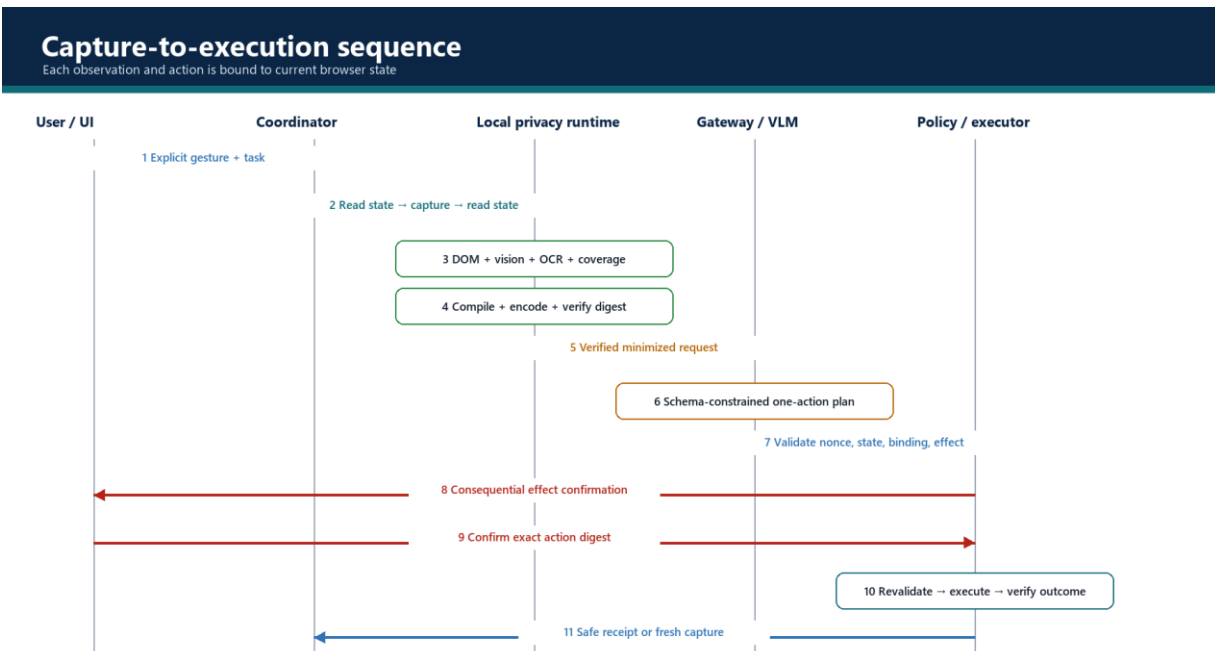


Figure 4. Capture-to-execution sequence with fresh-state validation.

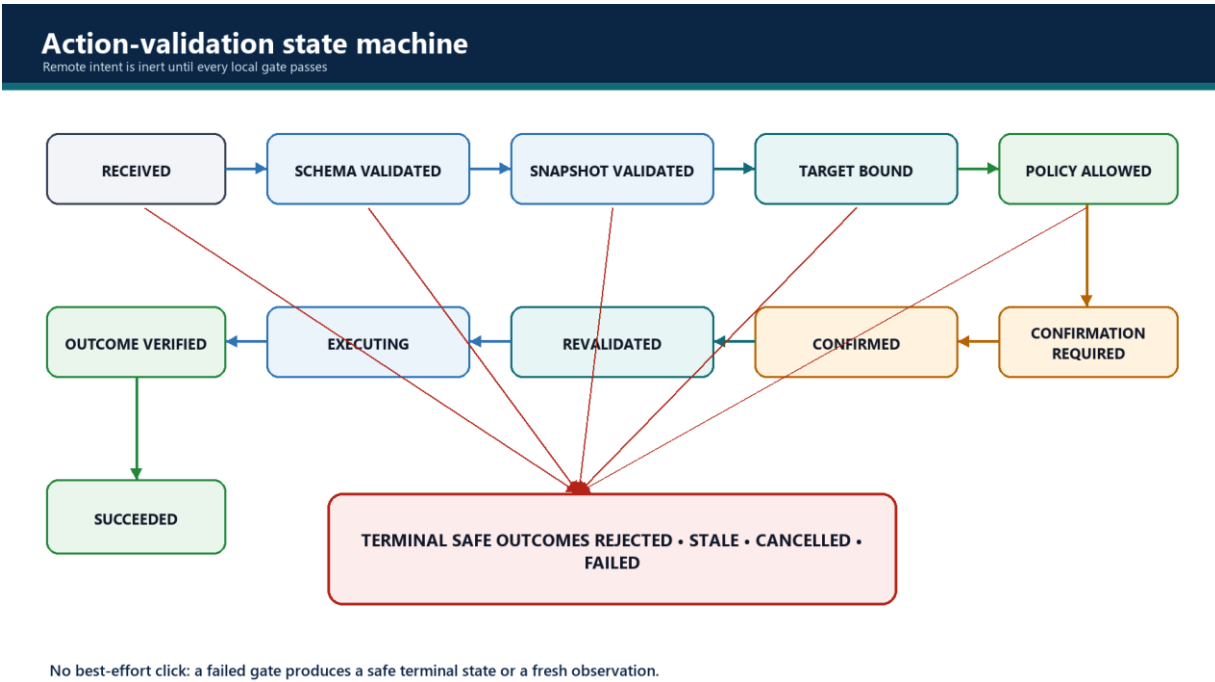


Figure 5. Local action-validation lifecycle and safe terminal states.

8.1 Capture-to-execution sequence

19. User activates the extension and states a task.
20. Local task sanitizer classifies the task; forbidden secrets cause a prompt to rephrase locally.
21. Coordinator creates a task nonce, obtains temporary tab authority and reads page state.
22. DOM probe and visible-tab capture run; a second state read confirms consistency.
23. Coordinate normalization, detector, ROI OCR, PII ensemble and coverage analysis run locally.
24. Privacy compiler creates sanitized graph, typed token capabilities and a fresh image.
25. Final verifier inspects exact bytes and either blocks, remarks or creates a digest-bound envelope.
26. Gateway sends only the verified minimized request.
27. Remote VLM returns one action over an opaque element ID.
28. Local validator checks response, state, target and deterministic policy.
29. If consequential, the user confirms the exact digest in extension-owned UI.
30. Validator checks again, executor performs the DOM-backed action, and postconditions are observed.
31. A content-free receipt is stored locally; the next snapshot starts only if task policy permits.

8.2 Task state machine

The nominal path is:

```

IDLE -> AWAITING_USER_GESTURE -> CAPTURING -> PERCEIVING -> SANITIZING -> VERIFYING ->
AWAITING_REMOTE_PLAN -> VALIDATING_ACTION -> [AWAITING_CONFIRMATION -> VALIDATING_ACTION] ->
EXECUTING -> VERIFYING_RESULT -> CAPTURING or COMPLETED.

```

Rules:

- privacy or policy failure enters **BLOCKED**; there is no transition from **BLOCKED** directly to remote planning;
- stale but recoverable state cancels the current work and returns to **CAPTURING** with a new snapshot;

- availability failure enters **FAILED**; a user-initiated retry begins fresh;
- user decline, expiry, tab closure, vault loss or explicit stop enters **CANCELLED**;
- task state and browser permissions are tab-scoped.

8.3 Snapshot and privacy states

CURRENT may become **STALE_MUTATION**, **SUPERSEDED** or **INVALIDATED_NAVIGATION**. Terminal snapshot states never become current again.

Privacy state is:

UNVERIFIED -> **VERIFYING** -> **SAFE_TO_SEND**

or

UNVERIFIED -> **VERIFYING** -> **REQUIRES_ADDITIONAL_MASKING** -> **VERIFYING**.

Blocking terminals are **BLOCKED_LOW_COVERAGE**, **BLOCKED_RESIDUAL_PII** and **BLOCKED_POLICY**. A state called "safe after expanded mask" is intentionally avoided because an expanded artifact is not safe until reverified.

8.4 Navigation and token lifecycle

Any navigation invalidates the snapshot and element IDs. Unexpected or cross-origin top-level navigation cancels the task and destroys all token capabilities. For an expected same-origin workflow transition, page-derived tokens are destroyed and a new navigation-scoped vault is created. Only explicit user-task tokens that were locally authorized for multi-page use may be reissued with a new token ID, purpose and origin constraint.

9. Browser compatibility strategy

Capability	Chromium target	Firefox target	Safe fallback
Visible viewport capture	<code>tabs.captureVisibleTab</code> ; max 2/s [R2]	WebExtensions compatible API [R5]	Block unsupported/protected pages
Temporary page access	<code>activeTab</code> after gesture [R3]	<code>activeTab</code> on supported versions [R5]	Explicit optional host grant or unsupported
Background lifecycle	MV3 service worker	MV3-compatible adapter	Active extension compute host; vault loss cancels
Heavy inference	WebGPU when detected	Do not assume WebGPU parity	Certified WASM SIMD tier [R7]
Offscreen compute	Offscreen/side-panel adapter	Extension page/worker adapter	Foreground extension UI while task runs
DOM collection	Main + accessible same-origin frames	Same principle, API differences isolated	Mask/block inaccessible region
Package	WXT-generated MV3	WXT-generated compatible manifest	Separate manifest adapter, shared domain code

Browser support means tested behavior on a frozen matrix, not merely successful compilation. The hackathon claim should be "Chromium end-to-end; Firefox capability smoke test" unless both paths are measured.

10. Data contracts

The wire authority is Draft 2020-12 JSON Schema in `contracts/v1/drishtiguard.schema.json`; TypeScript types are generated/checked against it in `packages/contracts/src/v1.ts`. [CONTRACTS.md](#) defines field semantics.

Identifier corrections are important:

- `snapshot_id` identifies one coordinated DOM/pixel observation.
- `document_frame_id` identifies a top-level document or iframe.
- `document_id` identifies the browser document where available.
- `navigation_id` is a top-level navigation epoch.
- `mutation_epoch` is a monotonic page-change version.

Local-only types may contain opaque `RawBitmapHandle`, `RawDomHandle` or `ValueHandle` capabilities. No remote-bound schema can represent them. Public `SensitiveFinding` records class, geometry, source, confidence and policy only - never a value or stable value-derived hash.

`RemoteInferenceRequest` excludes tab IDs, raw host/path/query/fragment, DOM HTML, attributes not explicitly sanitized, selectors, raw screenshot references and token mappings. `ActionPlan` is a closed discriminated union and contains one action only. All schemas set `additionalProperties: false`, length/count limits and explicit enums.

11. Deployment topology and scaling

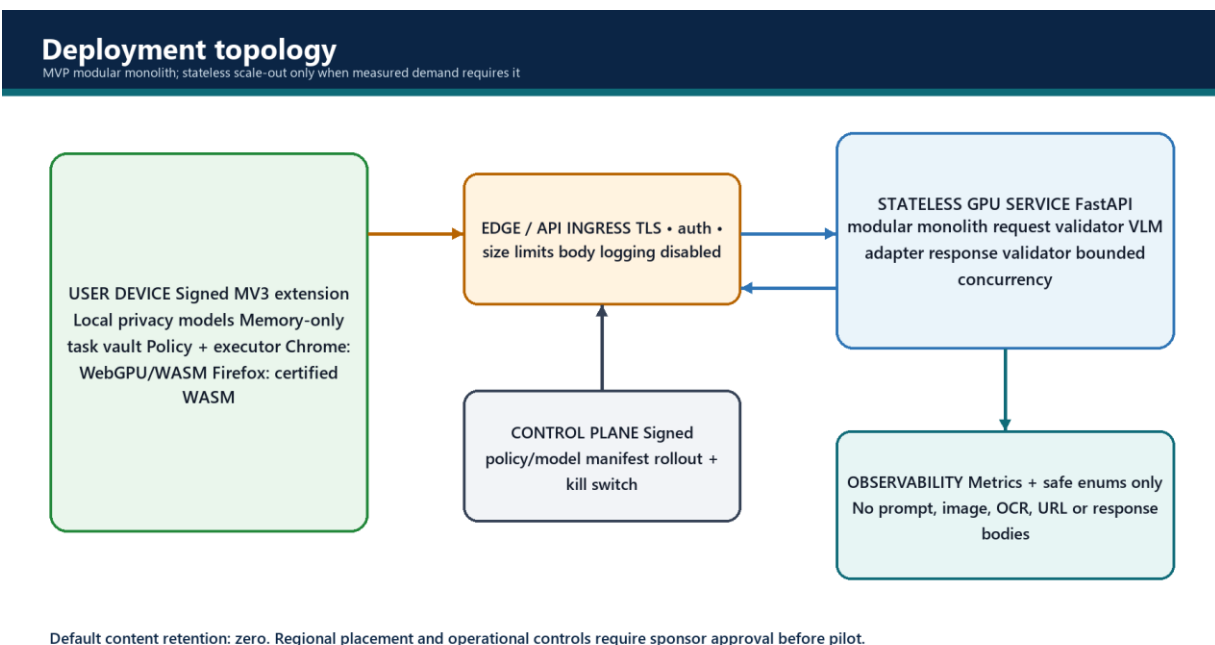


Figure 6. MVP deployment topology and content-free operational path.

11.1 Hackathon deployment

- Signed development extension loaded locally.
- Frozen bundled privacy models and hashes.
- One FastAPI service and one VLM runtime on a single GPU workstation or approved hosted GPU.
- HTTPS tunnel or local network TLS; short-lived runtime credential; no persistent request bodies.
- No Kubernetes, message bus, database or object storage.

11.2 Production target

- Store-signed extension releases with staged rollout and kill switch.
- TLS ingress, OIDC/PKCE-derived short-lived tokens, tenant/origin policy and per-user rate limits.
- Stateless service replicas with bounded queues and one-action requests; GPU-aware autoscaling by queue delay and memory.
- No screenshot/object store. Control-plane storage contains only policy/version metadata.
- Content-free metrics and alerts in a separate account/project; diagnostic uploads remain explicit and separately consented.
- Model manifest signing, hash verification, canary deployment and rollback.

Scale is horizontal at the stateless request layer and by GPU replica pools. Queue depth is capped; overload returns `REMOTE_CAPACITY` rather than keeping a page snapshot alive indefinitely. Requests are idempotent by `request_id` and late responses are rejected locally.

12. Performance architecture

The design reuses model sessions, lazily loads models after activation, warms one selected tier, batches OCR crops, hashes stable regions, performs full-frame pixel differencing, cancels stale work and uses transferable buffers. A changed-region result is valid only when navigation, viewport, zoom and dynamic-surface policies permit reuse; final outbound verification always covers the complete payload.

Targets, not results:

Measure	Target	Caveat
Store extension package including MVP models	<= 50 MB compressed	Must report downloaded cache and decoded resident weights separately
Peak incremental memory	<= 350 MB	Include JS/WASM heap, image buffers and GPU/process memory
Chromium/WebGPU local p95	<= 1.2 s	Ambitious on low-end hardware with OCR
Firefox/WASM local p95	<= 2.0 s	Requires small ROIs and sequential models
End-to-end warm p95	<= 4.0 s	Depends on network region, GPU queue and VLM tokens
Main-thread long task	<= 50 ms	Worker transfer and UI instrumentation required

If a certified fallback cannot meet privacy coverage, the step blocks. Latency pressure never bypasses verification.

13. Failure handling

Failure	Required behavior	Safe error family
Permission denied or expired	No capture/send; return to user gesture	PERMISSION_*
Protected or unsupported page	Stop; do not reuse prior frame	CAPTURE_UNSUPPORTED
Capture/DOM state mismatch	Discard and recapture after debounce	SNAPSHOT_STALE
Invalid coordinate transform	Discard; never mask or click with uncertain mapping	COORDINATE_*
Cross-origin/closed-shadow/canvas uncertainty	Mask whole region, block or require user-assisted mode	COVERAGE_OPAQUE
WebGPU unavailable	Use prevalidated WASM tier	RUNTIME_WEBGPU_UNAVAILABLE
WASM init/OOM/model hash failure	Smaller certified tier or block	RUNTIME_* / MODEL_*
OCR/PII timeout	Coverage incomplete; mask unresolved area or block	COVERAGE_TIMEOUT
Low confidence or detector disagreement	Union and expand; unresolved critical case blocks	PRIVACY_LOW_CONFIDENCE
Encoder or metadata failure	Discard artifact and block	PRIVACY_ENCODING
Residual PII	Bounded remask/reverify; persistent result blocks	PRIVACY_RESIDUAL
Vault expiry/reload/update	Destroy handles and cancel task	TASK_VAULT_LOST
Gateway schema/digest/policy failure	Reject before network I/O	EGRESS_*
Remote timeout/network failure	No action; retry only while snapshot remains current	REMOTE_*
Invalid, replayed or late response	Reject without execution	ACTION_RESPONSE_*
Prompt-injected forbidden action	Reject locally and record safe outcome	ACTION_POLICY
Element moved/disappeared/changed	No-op, recapture and replan	ACTION_STALE
Confirmation declined or expired	No-op; receipt only	ACTION_CANCELLED

Failure	Required behavior	Safe error family
Unexpected post-action navigation	Stop chain and re-observe with user authority	<code>ACTION_POSTCONDITION</code>
Telemetry scrub failure	Drop telemetry, never raw fallback	<code>TELEMETRY_DROPPED</code>

Every row becomes an automated fault-injection case.

14. Observability without page content

Permitted events contain timestamp bucket, random trace ID, component, stage duration, browser capability flags, execution provider, model/policy/schema versions, finding counts by class, mask count and area ratio, payload byte count, decision enum, safe error code, memory/CPU buckets and cold/warm flag.

Forbidden fields include screenshots, image snippets, OCR/DOM text, raw or sanitized personal values, token mappings, user task, URLs beyond a local origin alias, prompts, model responses, selectors, HTML, stack-local variables and request/response bodies.

Diagnostics are off by default for the MVP. Pilot diagnostics are opt-in, short-retention and processed through a second fixed telemetry schema. If the scrubber fails, the event is dropped. Privacy tests place synthetic canaries in every forbidden channel and scan browser storage, console, crash output, proxy logs, service logs and telemetry sink.

15. Security and supply chain

- Strict extension CSP; no inline/eval/remote code; endpoint allowlist.
- Exact dependency lockfiles, automated SCA, SBOM, license inventory and review of transitive packages.
- Store-signed extension and signed release provenance; two-person approval for privacy/gateway changes.
- Model files addressed by SHA-256 in a signed manifest; verify before session creation; quarantine corrupt cache.
- Reproducible builds where practical; compare artifact hashes in CI.
- TLS for all remote communication; certificate validation; short-lived bearer tokens; no private client key.
- Server authentication, per-principal/origin rate limits, body limits, timeouts and strict JSON Schema at both boundaries.
- Image decoder hardening and decompression limits.
- Feature flags that can only reduce capability or block; rollback of model, policy and service.
- Security tests for message spoofing, replay, DOM clobbering, prototype pollution, prompt injection, request smuggling and stale actions.
- No page-derived content in CI artifacts, crash capture, APM or debug recordings.

Downloaded model weights may be treated as remotely supplied behavior by browser-store policy.

Production must confirm store acceptance; otherwise models ship inside reviewed extension releases.

Models are data to the runtime, but still part of the trusted supply chain.

16. Prompt-injection defense

AgentDojo demonstrates that untrusted tool data can hijack agents and provides realistic security test cases [R14]. DrishtiGuard layers controls:

32. webpage text is labeled untrusted observation, never system or user authority;
33. remote prompts use separate immutable channels and a small closed action vocabulary;
34. model output carries no capability by itself;
35. the local policy engine rejects secret access, uploads, permission changes, cross-task requests and arbitrary destinations;
36. task scope and origin scope are local, not model-editable;
37. high-risk effects require exact-action confirmation;
38. target/state checks run after the user confirms;
39. prompt-injection variants are tested in visible text, hidden DOM, ARIA labels, image text, canvas and QR codes.

The system should show a poisoned page asking it to upload a document and demonstrate a local `ACTION_POLICY` rejection.

17. Rollout

17.1 Hackathon MVP

Implement Chromium end-to-end on the synthetic portal: explicit side panel; same-origin DOM targets; one canvas/image hard case; DOM + visual/OCR PII fusion; task sanitizer; coverage map; typed random tokens; solid-mask fresh bitmap; exact-payload verification; strict one-action remote service; submit confirmation; stale-plan and prompt-injection rejection; intercepted-request evidence; measured latency, memory and confusion counts. Firefox receives a capability and smoke-test build unless time permits measured parity.

17.2 Post-selection engineering

Freeze benchmark/generator versions; calibrate Indian PII and multilingual cases; harden model delivery; complete Firefox behavior; add dynamic-page correctness; authentication, rate limiting and safe telemetry; broaden fault injection; generate TypeScript from the schema; and establish signed release provenance.

17.3 Pilot readiness

Restrict to allowlisted portals and workflows; add reviewed site adapters; obtain retention/residency approval; integrate OIDC/PKCE and enterprise policy; run external security/privacy review and prompt-injection red team; define SLOs, incident response and GPU capacity; complete store/managed-deployment review.

17.4 Production readiness

Require independent audit, statistically adequate per-class benchmarks, sustained browser regression, residual-risk acceptance, model/supply-chain governance, managed canary/rollback, incident exercises and measured cost/capacity. The MVP must never be renamed production-ready without these gates.

Detailed milestones and exit criteria are in [IMPLEMENTATION ROADMAP.md](#).

18. Open decisions

Decision	Recommended default	Owner / deadline
Exact team/time and skills	Six-person split; revise immediately	Team lead, before sprint
Required browser versions	Latest two stable at freeze	Extension lead, day 0
Target demo hardware	16 GB Windows laptop, record exact CPU/GPU	ML/perf lead, day 0
Remote hosting/self-host mandate	Self-hosted approved India region	Sponsor, before pilot
Official PII classes	Use conservative proposed set	Privacy owner, before corpus freeze
Data residency and retention	India + zero content retention	Sponsor/legal, before pilot
Authentication	Short-lived demo token, PKCE later	Security owner
Enterprise policy	Out of MVP, allowlisted pilot	Sponsor
Expected volume/SLO	Unknown; cap concurrency	Service owner before load test
Sanitized image retention	Never by default	Privacy owner
Model-update delivery policy	Bundled for MVP; signed pack only if store-approved	Release owner
Public source release	After secret/license/security review	Team + sponsor
Amount/financial-value policy	Send derived facts, not exact value	Product/privacy owner

19. Consistency review and hard truths

The following contradictions in the initial brief are resolved here:

- The user task is sanitized along with the screenshot and scene graph.
- `frame_id` is split into snapshot and browser-document-frame identities.
- Additional masking is non-sendable until a new verification pass succeeds.
- The in-memory vault lives in an active compute host; MV3 host loss cancels the task.
- Navigation invalidates element IDs and page-derived tokens; controlled continuity is explicit.
- Changed-region inference is only an optimization and cannot hide dynamic pixel changes.
- Canvas/cross-origin/PDF visual understanding does not imply reliable programmatic execution.
- A `CLICK` is not inherently low risk; policy evaluates effect.
- Confirmation is followed by revalidation to close the user-delay race.
- A manifest claiming "zero raw bytes" is not proof; the demo must inspect actual transmitted and server-received bytes.
- Financial information cannot be simultaneously declared sensitive and sent because it is useful; derive a safe predicate locally.

- The 50 MB target distinguishes compressed package, cached model files and decoded resident memory. The targets remain ambitious. An 85% grounding result is plausible on the declared synthetic portal family, not the open web. A 97% critical-class recall claim needs hundreds of cases per class and confidence intervals. "Zero leakage" must be stated as 0 of N on a frozen suite. The 1.2/2.0 second local p95 targets may fail on low-end devices with OCR. The four-second end-to-end target depends on a warm, nearby GPU with little queueing.

20. Blunt verdict

This architecture is strong enough for a credible SIH prototype if the team implements the privacy compiler, exact-payload verifier, intercepted-network evidence, stale-action refusal, prompt-injection rejection and user-confirmed submit path. A polished mockup without those controls is not credible.

The likely winning demo is not a universal browser agent. It is a narrow workflow that proves five things live:

40. DOM-only, vision-only and fused detections differ in a useful way.
41. Typed tokens preserve task meaning while raw values are absent from the actual request.
42. A deliberately seeded leak is caught by the second pass and blocked.
43. A stale or malicious action is refused locally.
44. Measured accuracy, memory, bundle size and latency are shown on the named laptop.

That is a defensible production-target architecture and an achievable prototype. It is not a production-ready implementation.

21. Architecture decision records

The following accepted ADRs form part of this baseline:

- [ADR-0001: DOM and vision fusion](#)
- [ADR-0002: WebGPU with WASM fallback](#)
- [ADR-0003: Typed token capabilities](#)
- [ADR-0004: Fail-closed final-byte verification](#)
- [ADR-0005: Remote planning, local execution](#)
- [ADR-0006: In-memory token vault](#)
- [ADR-0007: Cross-browser adapter layer](#)
- [ADR-0008: Solid masks, never blur for critical data](#)
- [ADR-0009: Changed-region inference](#)
- [ADR-0010: Privacy-safe telemetry](#)

22. References

- [R1] Smart India Hackathon, "SIH 2026 Problem Statements," live official page, verified 4 September 2026: <https://sih.gov.in/sih2026PS>
- [R2] Chrome for Developers, `chrome.tabs`, including `MAX_CAPTURE_VISIBLE_TAB_CALLS_PER_SECOND = 2`: <https://developer.chrome.com/docs/extensions/reference/api/tabs>
- [R3] Chrome for Developers, "The activeTab permission": <https://developer.chrome.com/docs/extensions/develop/concepts/activeTab>

- **[R4]** Chrome for Developers, Manifest V3 content security policy: <https://developer.chrome.com/docs/extensions/reference/manifest/content-security-policy>
- **[R5]** MDN, `tabs.captureVisibleTab()`: <https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/tabs/captureVisibleTab>
- **[R6]** WXT documentation: <https://wxt.dev/>
- **[R7]** ONNX Runtime Web support matrix and execution providers: <https://onnxruntime.ai/docs/get-started-with-javascript/web.html>
- **[R8]** PaddleOCR OCR pipeline and mobile models: https://www.paddleocr.ai/main/en/version3.x/pipeline_usage/OCR.html
- **[R9]** Microsoft Presidio, mechanisms and detection limitation: <https://microsoft.github.io/presidio/>
- **[R10]** Wang et al., "GUIGuard: Toward a General Framework for Privacy-Preserving GUI Agents," arXiv:2601.18842: <https://arxiv.org/abs/2601.18842>
- **[R11]** Zhao et al., "Anonymization-Enhanced Privacy Protection for Mobile GUI Agents," arXiv:2602.10139: <https://arxiv.org/abs/2602.10139>
- **[R12]** "CAPED: Context-Aware Privacy Exposure Defense for Mobile GUI Agents," arXiv:2606.12666: <https://arxiv.org/abs/2606.12666>
- **[R13]** Qwen, Qwen3-VL-4B-Instruct model card and Apache-2.0 license: <https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct>
- **[R14]** Debenedetti et al., "AgentDojo," NeurIPS 2024: https://proceedings.neurips.cc/paper_files/paper/2024/hash/97091a5177d8dc64b1da8bf3e1f6fb54-Abstract-Datasets_and_Benchmarks_Track.html
- **[R15]** OSU NLP Group, Mind2Web repository and dataset license: <https://github.com/OSU-NLP-Group/Mind2Web>
- **[R16]** Li et al., "ScreenSpot-Pro," arXiv:2504.07981: <https://arxiv.org/abs/2504.07981>
- **[R17]** GUIGuard-Bench public dataset card and CC BY-NC 4.0 license: <https://huggingface.co/datasets/ShaofantuoshuzhengzhiSha/GUIGuard-Bench>
- **[R18]** UIDAI, "What is Masked Aadhaar?": <https://www.uidai.gov.in/en/283-faq/aadhaar-online-services/e-aadhaar/1887-what-is-masked-aadhaar.html>
- **[R19]** Income Tax Department, PAN format guidance: <https://www.incometax.gov.in/iec/foportal/node/11593>
- **[R20]** Reserve Bank of India, IFSC definition: https://systemhealth.rbi.org.in/Scripts/FAQView.aspx_Id%3D60%281%29.html
- **[R21]** MeitY, Digital Personal Data Protection Rules, 2025: <https://www.meity.gov.in/documents/act-and-policies/digital-personal-data-protection-rules-2025-gDOxUjMtQWa>
- **[R22]** PaddlePaddle, PaddleOCR.js browser package and worker/runtime documentation: <https://github.com/PaddlePaddle/PaddleOCR/tree/main/paddleocr-js>

This document is an engineering architecture, not legal advice. DPDP applicability, notices, lawful purpose, processor terms, incident duties, residency and retention require counsel and sponsor review before a pilot [R21].

DRISHTIGUARD ARCHITECTURE BASELINE

Part II — Threat model

STRIDE-informed abuse cases, prompt injection, supply chain and residual-risk acceptance

Controlled source: docs/THREAT_MODEL.md

Version: 1.0

Date: 4 September 2026

Method: STRIDE-informed abuse-case analysis with privacy and agent-safety extensions

Scope: Browser extension, local privacy runtime, remote planning service, release pipeline and synthetic demo portal

1. Security objectives

DrishtiGuard must preserve confidentiality of page-derived data, integrity of local policy and actions, authenticity of extension/service messages, availability without privacy downgrade, and auditable behavior without content leakage.

The most important boundary claim is structural: the outbound gateway cannot accept original capture handles or the token vault. Detection of unknown sensitive meaning remains probabilistic and is represented as residual risk, not hidden behind an absolute guarantee.

2. Protected assets

Asset	Confidentiality	Integrity / availability concern
Raw screenshot and DOM/task text	Critical	A stale or modified snapshot can misground masks/actions
Credentials, OTPs, session tokens, IDs and financial/medical facts	Critical	Must never be intentionally collected for remote use
Token vault and token capabilities	Critical	Theft enables re-identification or unauthorized local fill
Browser permissions and task capability	High	Abuse can capture or modify an unintended tab/origin
Snapshot/navigation/mutation state	High	Tampering creates privacy gaps and TOCTOU actions
Privacy policy and detector configuration	High	Weakening can create silent egress
Sanitized artifact and verification digest	High	Substitution after verification bypasses the firewall

Asset	Confidentiality	Integrity / availability concern
Action plan, target binding and confirmation	High	Manipulation can cause consequential unintended effects
Extension/model/service binaries	High	Compromise subverts every higher-level control
Logs, crash dumps and telemetry	High	Secondary exfiltration channel
Availability and rollback controls	Medium	Outage must not trigger a weaker privacy mode

3. Trust boundaries

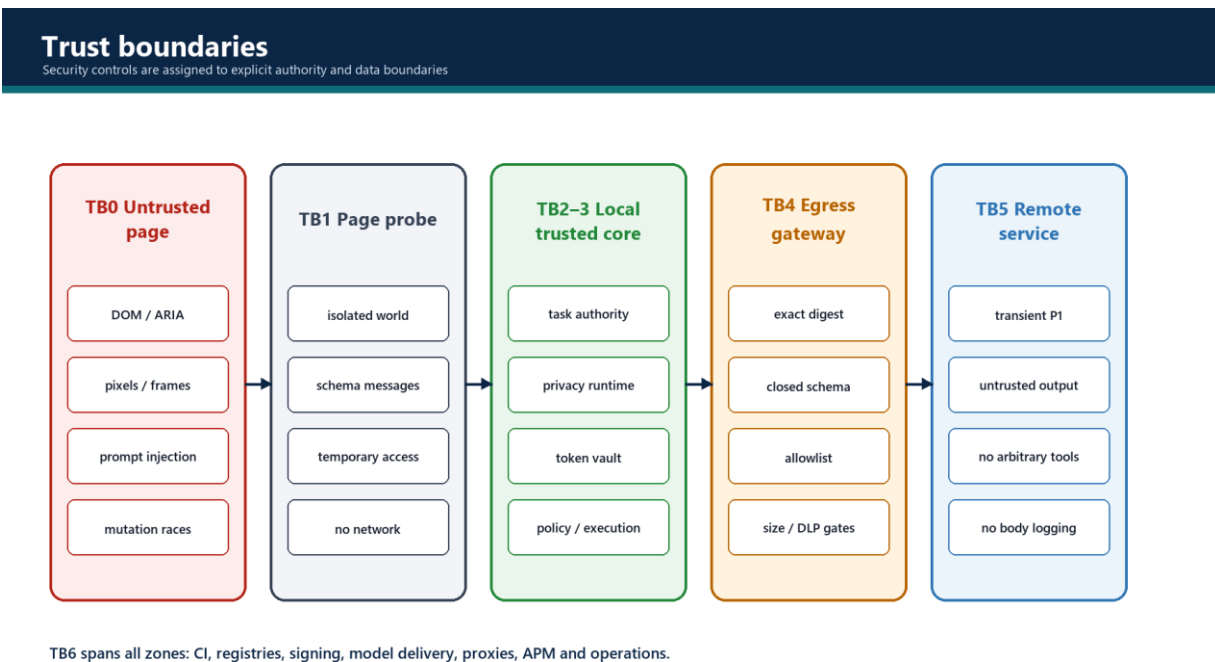


Figure 7. Trust boundaries used by the threat model.

- **TB0:** The webpage, including same-origin scripts, DOM/ARIA metadata, frames, text, images, canvas, QR codes and timing behavior, is hostile input.
- **TB1:** The isolated content script has temporary read/action access but no screenshot, vault or network capability.
- **TB2:** The extension core owns task authority, permissions, policy and action execution. It trusts only schema-valid messages tied to a current nonce.
- **TB3:** The local compute host handles raw pixels/text, model runtimes, masking, verification and the memory-only vault. Loss cancels the task.
- **TB4:** The gateway accepts only the exact digest approved by the verifier and only an allowed destination.
- **TB5:** TLS protects transport, but the remote service and model output are not trusted to make local policy decisions.

- **TB6 (supply chain/operations):** Extension store, CI, package registries, model host, signing keys, server image registry, proxies, APM and administrators are part of the operational attack surface.

4. Threat actors

- A malicious webpage author using prompt injection, deceptive controls, DOM clobbering, mutation races, invisible text, canvas, frames or data URLs.
- A compromised third-party page or advertisement in an otherwise trusted origin.
- A network attacker attempting interception, replay, downgrade or endpoint substitution.
- A compromised or malicious remote VLM/service returning unsafe or malformed plans.
- A curious service operator or telemetry administrator seeking content.
- A malicious or compromised extension dependency, model file, build runner, update channel or developer account.
- Another extension or local process attempting to read browser memory/storage or spoof messages.
- An authorized user making an error, confirming the wrong effect or enabling an overly broad origin.
- A denial-of-service actor triggering capture storms, adversarial images, expensive OCR or GPU queue exhaustion.

Local administrator compromise, kernel/browser compromise, physical memory forensics and malicious enterprise root certificates are outside the initial guarantee. They remain explicit residual risks.

5. Attack surfaces

- Extension action, side panel, content scripts, runtime messaging and browser APIs.
- Visible-tab capture, image decode/encode, OCR/model inputs and cross-frame coordinate transforms.
- DOM/ARIA attributes, form values, shadow roots, URLs and mutation events.
- Token creation/resolution and task/navigation lifecycle.
- Gateway serialization, multipart construction, endpoint configuration and authentication.
- Remote HTTP ingress, proxy/APM, prompt template, VLM runtime and output parser.
- Local execution, form events, navigation and confirmation UI.
- Browser storage, console, crash capture, traces and metrics.
- NPM/Python/container dependencies, WASM, ONNX weights, model download/cache and signing pipeline.

6. STRIDE and privacy analysis

ID	Category	Threat / abuse case	Primary controls	Detection / test	Residual risk
T01	Spoofing	Page script sends a message pretending to be the trusted probe	Isolated world, <code>runtime.id</code> /sender checks, task nonce, role, sequence, schema	Forged/replayed message tests	Browser/extension compromise defeats boundary
T02	Spoofing	Attacker substitutes the remote endpoint or service identity	Fixed allowlist, TLS validation, signed config, short-lived audience-bound token	DNS/proxy fault tests, certificate errors	Malicious enterprise trust root
T03	Tampering	Page mutates between DOM collection, screenshot, mask and click	Read-capture-read state check, mutation epoch, cancellation, pre/post-confirm validation	Mutation/scroll/zoom/navigation race suite	Mutation not observable before a synchronous effect
T04	Tampering	Encoded payload changes after verification	Immutable buffer ownership, SHA-256 envelope, gateway recomputation	Flip-byte and multipart-order tests	Compromised gateway code

ID	Category	Threat / abuse case	Primary controls	Detection / test	Residual risk
T05	Tampering	Model/policy file is replaced	Signed manifest, pinned hash, quarantine, rollback	Corrupt/cache-substitution tests	Signing-key compromise
T06	Repudiation	Consequential action has no trustworthy consent record	Extension-owned confirmation bound to action digest; safe receipt	Confirmation expiry/replay tests	User can still misunderstand sanitized description
T07	Information disclosure	Raw screenshot or DOM is sent accidentally	Non-serializable raw handles, gateway-only networking, lint/CSP, final-byte DLP	Intercept actual client/server bytes with canaries	Fully compromised extension
T08	Information disclosure	Screenshot is masked but task/ARIA/URL/action argument leaks value	Cross-modal privacy compiler, forbidden-field schema, task sanitizer	Same canary in every modality	Unknown semantic inference from safe-looking context
T09	Information disclosure	Blur/pixelation or alpha/metadata reveals content	Solid masks, fresh flattened image, fixed encoder, metadata/alpha check, adversarial OCR	Resize/contrast/crop/OCR and metadata suite	Unseen side channels in codec/runtime
T10	Information disclosure	PII detector misses a value, face or confidential fact	Ensemble, coverage map, union masking, unknown-region block, enterprise rules	Per-class/surface recall, adversarial cases	Inherent probabilistic false negatives
T11	Information disclosure	Token strings are guessed or page injects <code>[PAN_1]</code>	Random opaque token capability, task/origin/field/purpose binding; display label never resolves	Guess, collision, cross-task and literal-placeholder tests	Local memory theft
T12	Information disclosure	Vault survives task/navigation or leaks through storage	Memory-only compute host, TTL, explicit invalidation, no serializer	Tab close/reload/update/navigation storage scan	JS strings cannot be guaranteed physically zeroized
T13	Information disclosure	Proxies, APM or exception logs retain sanitized/raw body	Request-body logging disabled at every hop, content-free errors, deployment audit	Seeded log/crash scan	Operator changes configuration after audit
T14	Denial of service	Page creates capture/mutation storm	Debounce, two/s cap, one job per tab, cancellation, work budget	Mutation-flood tests	User-visible loss of automation, safely
T15	Denial of service	Crafted image/model input causes OOM or long inference	Pixel/ROI limits, decoder bounds, timeouts, tiered certified runtime	Oversize/decompression/adversarial image tests	Device-specific driver failure
T16	Elevation of privilege	Prompt injection requests secret access, upload or policy change	Observation channel, closed action union, local effect policy, no model tools	AgentDojo-derived and visual injection suite	Novel attacks may reduce task utility
T17	Elevation of privilege	Remote returns JavaScript, selectors, coordinates or arbitrary URL	<code>additionalProperties: false</code> , discriminated action schema, server and client validation	Fuzz/unknown-action tests	Parser/library vulnerability
T18	Elevation of privilege	Benign-looking click submits/deletes/pays	Effect-based risk, form/ancestor context, confirmation, revalidation	Deceptive-label and hidden-form tests	Semantic ambiguity leads to block/manual mode
T19	Elevation of privilege	Cross-origin iframe/canvas target is clicked incorrectly	MVP DOM-backed execution only; inaccessible targets user-assisted	Opaque-surface action tests	Reduced coverage and utility
T20	Repudiation / replay	Old action plan is reused on a new page	Request nonce, snapshot/navigation/mutation binding, expiry, one-time action ID	Replay and delayed-response tests	Clock anomalies; use monotonic expiry locally
T21	Tampering	Dependency/model update weakens privacy	Lockfile, SBOM, signed hashes, benchmark gates, two-person review, canary rollout	Reproducible build and regression suite	Upstream backdoor not detected by tests
T22	Information disclosure	Telemetry or debugging exports raw content	Fixed content-free schema, local-only debug, explicit opt-in, drop on scrub failure	Storage/console/collector canary scan	Manual developer tools can expose local data
T23	Spoofing	Confirmation UI is imitated by the webpage	Browser extension-owned side panel/popup; no page overlay confirmation	UI-origin/phishing test	User may not distinguish extensions in general

ID	Category	Threat / abuse case	Primary controls	Detection / test	Residual risk
T24	Tampering	Coordinate confusion leaves pixels unmasked or clicks neighbor	Actual-dimension scale, bounds and transform tolerance, region padding	DPR/zoom/scroll/frame/transform matrix suite	Browser rendering edge cases
T25	Information disclosure	Sanitized context still enables re-identification by combination	Data minimization, scene-graph-first, derived facts, policy thresholds	k-anonymity-style uniqueness heuristics and review	Contextual inference cannot be eliminated universally

7. Prompt injection and malicious webpages

Webpage text is data, never instruction authority. The prompt constructor places it in a delimited `UNTRUSTED_OBSERVATION` channel. It cannot change allowed origins, token resolution, privacy rules, action policy or confirmation requirements.

The remote model has no direct browser tool. A returned plan must survive:

45. strict schema and size validation;
46. task/request/snapshot nonce validation;
47. local task and origin scope;
48. live target binding;
49. deterministic effect policy;
50. user confirmation when required; and
51. a second state/target validation.

Tests place hostile instructions in visible text, tiny text, hidden DOM, accessibility names, image pixels, canvas, PDF-like images, QR codes and cross-origin frames. Required negative outcomes include attempts to upload a file, read a password/OTP, change permissions, navigate to an arbitrary origin and submit outside the stated task.

AgentDojo is used to inspire policy and indirect-injection scenarios, not as a visual-grounding benchmark [R14].

8. Extension compromise

If trusted extension code is fully compromised, the gateway abstraction alone cannot protect data because the attacker executes inside the trusted computing base. Defense is preventive and detective:

- least privilege and explicit activation;
- no remote executable code;
- restrictive CSP and isolated worlds;
- exact locks, SCA/SBOM, signed builds and store distribution;
- model/runtime hash verification;
- two-person review of capture, privacy and networking code;
- static rules that forbid network APIs outside adapters;
- reproducible artifact comparison and staged rollout;
- emergency kill switch that disables remote egress;
- continuous canary-based privacy regression.

The kill switch must only reduce capability. It cannot activate a less private pipeline.

9. Server and network compromise

TLS, authentication and rate limiting address interception and abuse, but the remote service is assumed potentially curious or compromised. It receives no raw token map and cannot directly act. Local validation and confirmation remain effective if the model is malicious.

The service must not include URL-fetch, browser, filesystem or shell tools. It rejects oversized images, malformed multipart bodies, unknown schema versions and replayed request IDs. Its reverse proxy, framework, model server and APM disable body logging. Secrets reside in server secret management, not extension bundles or images.

Server compromise can still expose the sanitized request and infer sensitive context. That is why the preferred request is scene-graph-only and the privacy model treats sanitized context as confidential derived data even when it is permitted for transient processing.

10. Screenshot and token-map leakage

Raw image bytes use linear ownership: capture adapter -> local worker -> privacy compiler -> zeroization/dereference. They are never written to IndexedDB, local storage, extension storage, cache, clipboard, console or error reports. Transferable buffers avoid copies where possible.

The token vault uses random 128-bit or stronger IDs, independent per task. Entries contain type, value buffer/handle, task, purpose, origin/field constraint, navigation scope, expiry and remaining uses. Display counters are not capabilities. Token resolution occurs only inside the executor after an allowed `token_ref` action and immediately before setting an approved field.

JavaScript garbage collection and immutable strings prevent a claim of guaranteed physical zeroization. The design minimizes lifetime/copies and overwrites mutable buffers, but local memory forensics remains out of scope.

11. Supply-chain controls

Layer	Required control
NPM/Python	Exact lock, registry integrity, dependency review, SCA, SBOM, license gate
Extension build	Hermetic/reproducible build where practical, protected signing, store signature
WASM/runtime	Packaged version, CSP-compatible build, hash and provenance
ONNX/OCR/VLM weights	Signed model manifest, SHA-256, license record, benchmark attestation
Container	Minimal base image, digest pin, scan, non-root, read-only filesystem where possible
CI/CD	Protected branch, two-person approval, short-lived credentials, artifact provenance
Configuration	Signed allowlist and policy, monotonic version, safe rollback
Operations	Least-privilege IAM, content logging disabled, audit of config drift

12. Residual risk register

Risk	Why it remains	Treatment
Unknown PII/confidential data is missed	Recognition is probabilistic and domain-dependent	Measure per class/surface; conservative coverage and site policy; sponsor accepts residual risk
Sanitized context enables inference	Combinations may uniquely identify a person or organization	Minimize fields, send derived facts, scene-graph first, no retention
Local device/browser compromise	Attacker is inside the trusted boundary	State as out of scope; enterprise hardening for pilot
Confirmation misunderstanding	Sanitization can obscure context	Clear effect summary, origin alias, manual mode for ambiguity
Cross-origin/canvas/PDF utility gap	Standard extension APIs cannot bind all visual targets	Mask/read/manual-assist or reviewed site adapter
Model adversarial examples	Detectors and VLMs can be manipulated	Diverse tests, union masking, coverage gate, local policy
Operational log drift	Proxy/APM settings can change	Config-as-code, drift audit, canary monitoring
Browser/runtime variance	API and GPU behavior differs	Frozen compatibility matrix and certified fallback

Residual risks must be accepted explicitly before pilot. Weighted SIH performance cannot waive a critical privacy or action-safety gate.

13. Security verification plan

- Contract fuzzing for every local/wire schema and unknown field.
- Static import graph proving only gateway/artifact/telemetry adapters have network APIs.
- Seeded canaries in pixels, task, DOM, ARIA, URLs, attributes, OCR and action arguments; scan actual client request, server ingress and all logs.
- Adversarial OCR after scale, sharpen, contrast, crop, format conversion and channel inspection.
- Message spoof/replay, permission expiry, service-worker eviction and vault-loss tests.
- Navigation, mutation, resize, zoom, frame and confirmation-delay races.
- Prompt-injection and deceptive-target scenarios adapted from AgentDojo.
- Dependency/model substitution, corrupt cache and rollback exercises.
- Fault injection for GPU/WASM/OCR/service/network failures, asserting no policy downgrade.
- Independent extension and privacy review before any pilot with real data.

14. Security acceptance criteria

- Zero raw seeded canaries in intercepted network requests, server logs, browser storage, console, crash output and telemetry for the declared suite.
- Zero execution of stale, replayed, schema-invalid or policy-forbidden plans.
- 100% confirmation enforcement for the declared high-risk action set.
- All critical PII classes meet their frozen per-class recall floor; no micro-average can hide a weak class.
- Every production artifact has verified provenance, dependency/model inventory and rollback target.
- Open critical/high findings block release unless the security owner and sponsor formally narrow scope.

DRISHTIGUARD ARCHITECTURE BASELINE

Part III — Privacy model

Data classification, token lifecycle, cross-modal sanitization, exact-byte verification and deletion

Controlled source: docs/PRIVACY_MODEL.md

Version: 1.0

Date: 4 September 2026

Default posture: Data minimization, transient local processing, exact-payload verification and fail-closed egress

1. Privacy promise and limitation

DrishtiGuard is designed so that original captured-image buffers and reversible token mappings have no representable path through the outbound gateway. Values that the local system discovers or classifies as sensitive are removed or replaced across every outbound modality.

The system does not claim universal detection of all sensitive meaning. Names without context, confidential project codes, steganography, unique combinations of public facts and adversarial visuals can evade probabilistic classifiers. That residual risk is reduced by coverage analysis, conservative policy, final-byte verification and restricted pilot scopes, then measured and disclosed.

2. Privacy principles

- **Local first:** Raw pixels, raw DOM/task text, OCR output and token mappings are processed only inside the user-device boundary.
- **Minimize before sanitize:** Prefer a sanitized scene graph or locally derived predicate over sending an image or exact value.
- **All modalities:** Image masks alone are insufficient; task, labels, attributes, URL components and action arguments follow the same policy.
- **Purpose binding:** A token is usable only for a declared task, origin, target field, purpose and short lifetime.
- **Exact artifact:** The verifier examines the final encoded image and serialized request and authorizes their digest, not an abstract intermediate.
- **Fail closed:** Incomplete coverage, residual critical PII, invalid metadata or policy uncertainty blocks egress.
- **No hidden retention:** Content retention is zero by default at the remote service and telemetry layer.
- **User agency:** Capture starts after an explicit gesture; consequential effects require a fresh, understandable confirmation.

3. Data classification

Class	Examples	Default handling
-------	----------	------------------

Class	Examples	Default handling
P0 - Public/declared safe	Generic role names, non-identifying UI type, policy-approved static labels	May enter sanitized scene if needed
P1 - Sensitive-derived	Typed placeholders, origin alias, counts, coarse layout, derived predicate	Remote permitted only after verification; still confidential and no-retention
P2 - Personal/confidential	Name, email, phone, face, address, employee ID, medical/financial fact, exact URL path	Local only; replace, generalize or mask
P3 - Critical secret	Password, OTP, session/auth token, card/CVV, private key, recovery code	Never intentionally read for agent reasoning; mask region and block any requested disclosure
O0 - Operational metadata	Component latency, model version, safe error enum, finding counts	Telemetry permitted through a separate fixed schema

Input is classified P2 until a local policy proves it safe or transforms it into P1. "Sanitized" does not mean public.

4. Data inventory and lifecycle

Data	Location	Lifetime	Egress	Logging
Raw screenshot buffer	Privileged capture -> local compute	One snapshot; overwritten/dereferenced immediately after compile	Never	Never
Raw DOM/ARIA/task text	Page probe/local compute	One snapshot/task sanitization	Never	Never
Raw OCR output/crops	Local OCR worker	One processing attempt; stable safe cache only after classification	Never	Never
ValueHandle / token map	In-memory vault in active compute host	10 min idle, 30 min hard max, or earlier invalidation	Never	Never
Sensitive finding geometry/class	Local pipeline	Current snapshot; public variant may enter manifest without value	Class/count only	Counts only
Sanitized scene graph	Local compute, transient remote request	Until response or timeout	Allowed after verification	No content
Sanitized image	Local compute and transient server memory	Until response or 60 s hard server timeout	Allowed only when needed and verified	Never retained
Verified envelope/digest	Extension memory	Request lifetime	Digest/IDs allowed	Safe fields only
Action plan	Server response/local validator	Current snapshot only	Returns to client	No content logging
Execution receipt	Local extension	Session by default; optional 24 h safe history	Never by default	Safe enums/latencies only
Telemetry event	Separate exporter/sink	Suggested 7 days for pilot, subject to approval	O0 fields only	It is the log record

The exact TTLs are recommended defaults, not sponsor-approved policy. Closing the tab, canceling the task, losing the compute host, extension update/reload, unexpected navigation or expiry destroys handles and cancels dependent work.

5. Collection and consent

The extension does not run continuous background capture. The user invokes the extension through its action, shortcut or extension-owned UI, granting temporary active-tab authority. Before first remote use on an origin, the UI explains:

- what visible content is captured locally;
- what categories are masked or tokenized;
- whether a sanitized image is required or a scene graph is sufficient;
- which remote service receives transient sanitized content;
- that automated recognition has residual error; and
- which effects require confirmation.

Consent is bound to task and origin scope. Cross-origin navigation, permission expiry or material task change requires a new decision. A preview mode shows the sanitized artifact before sending and is mandatory for the MVP demo.

High-risk action confirmation is separate from processing consent. It is displayed in the extension surface, references a sanitized effect and origin alias, expires in 30 seconds, and binds to the exact action digest.

6. Detection and redaction policy

6.1 Critical classes

Credentials, OTPs, authentication/session tokens, private keys, recovery codes, Aadhaar, PAN, bank account details, payment cards/CVV and policy-declared high-impact organizational secrets are critical. Critical uncertainty results in expanded solid masking or blocking. Blur and weak pixelation are never accepted.

6.2 Other sensitive classes

Names, faces, email, phone, employee ID, address, medical facts, financial amounts, invoice identifiers and documents are replaced by typed tokens, generalized into a safe derived fact, or masked according to task purpose. Exact values are not sent merely because they are useful; the local system computes predicates such as `within_policy_limit=true` when possible.

6.3 Detection sources

The canonical privacy set is the union of DOM/form metadata, local OCR, patterns/validation, NER, face/document/QR/barcode detection, nearby labels, user declaration and site/enterprise policy. A coverage map proves that every visible region was scanned, masked, explicitly safe by policy or marked unknown. Unknown image/canvas/frame regions do not pass by default.

6.4 Geometric policy

Text regions expand to line height and receive class-specific padding. Overlapping regions merge. For disagreement, use the larger union. Masks are drawn into a new opaque bitmap and then encoded with a fixed profile. The original image is not retained as a background with reversible overlays.

7. Typed token vault

A token has two representations:

- a human-readable display label such as `[PAN_1]`; and
- a random task capability such as `tkn_7Y9...` with at least 128 bits of entropy.

Only the random ID can be resolved. The vault entry binds:

- privacy class and raw value buffer/handle;
- task ID and user-authorized purpose;
- allowed origin alias and target element/field;
- creation navigation epoch;
- expiry and remaining use count; and
- provenance of the local finding.

A model-created token string, copied display label, cross-task ID or expired ID never resolves. Resolution happens only after policy, confirmation and post-confirmation freshness checks, immediately before a permitted local field operation.

The vault is memory-only in the active compute host. It is not put in `chrome.storage`, IndexedDB, local storage, service-worker event state, logs, traces or crash reports. If multi-page operation is approved, page-derived mappings are destroyed on navigation. A user-task token may be reissued with a new ID only after explicit purpose/origin reauthorization.

JavaScript cannot guarantee physical erasure of immutable strings. The implementation minimizes copies, uses mutable byte buffers where practical, overwrites them before release and treats local memory inspection as a residual risk.

8. Cross-modal sanitization

The compiler processes:

- screenshot pixels and image metadata;
- scene-graph labels and values;
- accessible names, `alt`, `title`, `placeholder`, `autocomplete` and selected attributes;
- URL path, query and fragment; raw origin is replaced with a local alias unless necessary;
- user task and local conversation context;
- local derived facts and action arguments; and
- error/debug fields.

Remote-bound schemas cannot express raw HTML, selectors, tab IDs, raw handles, token maps or arbitrary URLs. Any unknown field fails schema validation.

9. Verification and egress

The final verifier decodes and inspects the exact image bytes and parses the exact serialized JSON/multipart structure. Checks include known-value/canary search, critical patterns, secondary OCR, mask coverage, alpha/metadata, dimensions, forbidden keys, coverage completeness and payload size.

Possible outcomes are:

- `SAFE_TO_SEND`
- `REQUIRES_ADDITIONAL_MASKING`
- `BLOCKED_LOW_COVERAGE`
- `BLOCKED_RESIDUAL_PII`
- `BLOCKED_POLICY`

Only `SAFE_TO_SEND` produces a short-lived envelope containing the exact body digest. A recoverable residual triggers at most two redaction retries. The gateway recomputes the digest immediately before sending and rejects mismatches, expiry, stale snapshot, wrong destination or missing consent.

The preferred disclosure order is:

52. locally answer the task without remote processing;
53. send a sanitized semantic graph and derived facts only;
54. attach a downscaled verified sanitized image only if visual context is necessary; or
55. block / request user-assisted action.

10. Remote processing and retention

The recommended remote service is self-hosted in an approved India region or on-prem environment, pending sponsor direction. It receives transient P1 data over TLS. Reverse proxy, API framework, APM, exception reporting, model server and infrastructure logs all disable body capture.

Request/image/prompt/response content is held only in volatile memory for the request, with a 60-second hard processing timeout and immediate release after response. No object storage, prompt history, training reuse, cache, support replay or human review is enabled by default. Any future diagnostic upload requires a separate explicit workflow, minimization, purpose, retention and deletion control.

11. Telemetry and local debugging

Telemetry uses an independent O0 schema. Allowed examples are component latency, model/policy version, capability flags, finding counts by class, mask area bucket, payload byte count, decision enum, resource buckets and generic error code.

Screenshots, snippets, OCR/DOM/task text, URLs, token IDs/mappings, prompts, responses and stack locals are forbidden. Telemetry scrubbing failure drops the event. Local debug mode is explicit, visually indicated, expires automatically and still never writes raw values by default.

Synthetic canaries are scanned in browser storage, console, crash output, proxy/service logs and telemetry sink during every privacy regression run.

12. Deletion behavior

- **Task complete/cancel/expiry:** overwrite mutable buffers where possible; remove references, workers and vault entries.
- **Navigation:** invalidate snapshot/targets and destroy page-derived token entries; unexpected/cross-origin navigation cancels the task.
- **Tab close or compute-host loss:** cancel task and make every capability unusable.
- **Extension removal:** clear model cache, safe preferences and local receipt history.
- **Server:** release request memory after completion/timeout; no content record exists to delete under the default design.
- **Telemetry:** delete after approved retention, suggested seven days for pilot; expose a local opt-out/reset control.

13. Privacy invariants

56. Raw screenshot buffers are never accepted by a network-bound contract.
57. Reversible token mappings never leave the active local compute boundary.
58. Sensitive discoveries are transformed across all outbound modalities.
59. The exact outgoing bytes must have a current `SAFE_TO_SEND` digest.
60. Critical data uses solid, opaque, flattened masking, never blur.
61. Unknown/low-coverage regions are masked or block the image path.
62. User tasks are classified and sanitized before remote planning.
63. Token resolution is random-ID, task, origin, field, purpose, TTL and use-count bound.
64. Navigation invalidates snapshots, element IDs and page-derived mappings.
65. Telemetry never contains page content or raw/sanitized personal values.
66. Availability failure never selects a weaker privacy pipeline.
67. Test/demo data is synthetic and invalid against real registries.

14. Legal and policy posture

The architecture minimizes personal data and emphasizes notice, purpose, security, deletion and processor controls, but it is not a legal compliance determination. India's Digital Personal Data Protection Act and notified 2025 Rules have phased commencement; counsel and the sponsor must determine applicability, notices, lawful purpose, processor terms, security safeguards, incident duties, retention and cross-border constraints for the intended deployment [R21].

UIDAI, financial-sector and organizational rules may impose stricter handling. DrishtiGuard fully masks Aadhaar-class values for remote egress even though UIDAI defines a masked-Aadhaar presentation that reveals the last four digits [R18].

15. Privacy acceptance evidence

- Actual intercepted request and server-ingress bytes contain zero seeded raw canaries for the declared suite.
- Critical PII recall and residual leakage are reported per class and surface with sample size and confidence interval.
- A seeded residual survives pass one, is detected by final verification and results in remasking or blocking.

- No raw content appears in storage, logs, console, crash output or telemetry.
- The user can inspect the sanitized preview and decline transmission.
- The server deployment audit proves body logging and content retention are disabled at every hop.

"Zero observed leakage" must be reported as 0 of N declared cases. With zero failures, the approximate 95% upper bound is $3/N$; it is not proof of universal zero risk.

DRISHTIGUARD ARCHITECTURE BASELINE

Part IV — API and contract specification

Trust-aware local types, strict remote schemas, action unions and compatibility policy

Controlled source: docs/CONTRACTS.md

Schema family: `com.drishtiguard.contracts/1.0`

Wire authority: JSON Schema Draft 2020-12

Rule: Local raw capabilities and remote wire objects are separate types. Conversion is one-way through the privacy compiler and final verifier.

1. Contract governance

- Every object includes `schema_version` and uses `additionalProperties: false`.
- Network objects have explicit size/count/string limits and closed enums.
- IDs are random opaque identifiers. They do not encode tab, URL, selector, value or person.
- Time is RFC 3339 UTC on the wire; local freshness uses a monotonic clock.
- Coordinates use an explicit coordinate-space identifier and carry snapshot lineage.
- Unknown major versions fail closed. A newer minor version is accepted only if its exact schema is registered; no permissive unknown-field behavior.
- JSON Schema is validated on client egress, server ingress, server egress and client response.
- TypeScript is generated or checked from the same schema in CI; hand-edited drift blocks release.
- Sensitive fields are not "optional" in wire types - they are impossible to represent.

2. Trust-aware type split

Type	Boundary	May contain raw content?	Serializable remotely?
<code>FrameSnaps hotLocal</code>	Privileged extension/local worker	Opaque raw bitmap and DOM handles	No
<code>RawDomElementLocal</code>	Page probe/local compute	Local text/value handles	No
<code>SensitiveFindingLocal</code>	PII ensemble/compiler	Non-serializable <code>SensitiveValueHandle</code>	No
<code>SensitiveFinding</code>	Sanitized internal/public	No value or stable value-derived hash	Yes only where included by policy
<code>SanitizedSceneLocal</code>	Compiler/verifier	Final encoded sanitized asset	No generic serialization
<code>VerifiedEnvelope</code>	Verifier/gateway	Digest and short-lived capability only	Gateway-internal

Type	Boundary	May contain raw content?	Serializable remotely?
RemoteInferenceRequest	Gateway/server	Sanitized P1 data only	Yes
ActionPlan	Server/client	Sanitized arguments or token refs only	Yes
ExecutionReceipt	Local audit	IDs, enums and timings only	Local/default no egress

`RawBitmapHandle`, `RawDomHandle` and `SensitiveValueHandle` are compile-time branded local capabilities whose constructors live inside trusted modules. Runtime serializers reject their prefixes and any object graph carrying a local-only brand; TypeScript branding alone is not a runtime control.

The canonical `FrameSnapshotLocal` fields are `origin_handle`, `raw_bitmap_handle` and `raw_dom_handle`. JSON Schema marks them `writeOnly` as an annotation; the trusted runtime serializer remains responsible for enforcing non-serialization.

3. Common identifiers

- `task_id`: one user-authorized task in one tab.
- `request_id`: one idempotent remote inference request.
- `snapshot_id`: one coordinated DOM/pixel observation.
- `document_frame_id`: top-level document or iframe identity.
- `document_id`: browser document identity where available; otherwise adapter-generated and navigation-scoped.
- `navigation_id`: top-level navigation epoch.
- `mutation_epoch`: monotonic material-change counter.
- `element_id`: task-random opaque UI target, invalid after navigation.
- `token_id`: task-random secret capability; display labels are never token IDs.
- `action_id`: one action response/attempt.

Runtime prefixes are type-specific: `tsk_`, `req_`, `snp_`, `nav_`, `doc_`, `frm_`, `elm_`, `fnd_`, `rgn_`, `tok_`, `act_` and `cor_`. JSON Schema rejects an otherwise well-formed ID used in the wrong identifier field.

The original brief's overloaded `frame_id` is replaced by `snapshot_id` and `document_frame_id`.

4. Geometry

`BoundingBox` is `{x, y, width, height}` in root visual-viewport CSS pixels unless another declared `coordinate_space` is used. Values are finite, non-negative and bounded by the declared viewport. Findings retain their source frame ID and normalized root box.

`Viewport` includes CSS width/height, scroll X/Y, screenshot pixel width/height, observed X/Y scale, DPR and browser zoom. The observed scale is derived from the actual captured image dimensions; DPR/zoom are diagnostic evidence.

5. FrameSnapshotLocal

Purpose: bind one raw capture and DOM observation to browser state.

Field	Type	Rule
schema_version	semver	1.0 family
task_id	opaque ID	Current local task
tab_id	browser integer	Local only
snapshot_id	opaque ID	New for every attempt
navigation_id	opaque ID	Changes on top-level navigation
document_id	opaque ID	Adapter source documented
mutation_epoch	integer	State before/after must match
origin_handle	local capability	Raw origin never enters remote request by default
captured_at	timestamp	RFC 3339 UTC; local monotonic freshness is held out of band
viewport	Viewport	Exact capture geometry
raw_bitmap_handle	local capability	Non-serializable
raw_dom_handle	local capability	Non-serializable

6. DomElementGraph

Purpose: sanitized semantic graph used for grounding.

Each `DomElement` contains `element_id`, optional parent ID, `document_frame_id`, role enum, `sanitized_label`, bounding box, visible/enabled/editable/checked/selected states, safe input category, source and confidence. It contains no selector, DOM path, raw attribute, value, HTML or URL. The local binding table maps `element_id` to a live target and stays inside the executor.

Limits: maximum 500 elements per scene in the MVP, label length 160, graph serialized size 256 KB. The compiler ranks by viewport visibility and task relevance before truncation; truncation that loses privacy coverage blocks, while truncation of remote grounding context is declared in the manifest.

7. VisualElement

Field	Meaning
element_id	Opaque element ID when fused to a DOM target; otherwise a non-executable observation ID
role	Closed semantic role such as button, input, text, icon, image, document, face or unknown
sanitized_label	Typed placeholder or verified safe label, max 160 characters

Field	Meaning
<code>bounding_box</code>	Normalized root viewport box
<code>document_frame_id</code>	Source document frame
<code>source</code>	DOM, VISION or FUSED
<code>confidence</code>	0..1; advisory, never sole privacy authority
<code>snapshot_id</code>	Exact observation lineage
<code>executable</code>	True only when a current DOM binding exists

8. SensitiveFinding

Field	Meaning
<code>finding_id</code>	Opaque random ID
<code>privacy_class</code>	Closed class enum
<code>sources</code>	One or more of DOM, OCR, VISION, PATTERN, NER, USER, SITE_POLICY
<code>bounding_box</code>	Optional for text-only task/metadata findings
<code>document_frame_id</code>	Optional source frame
<code>confidence</code>	0..1
<code>severity</code>	LOW, MEDIUM, HIGH, CRITICAL
<code>redaction_policy</code>	TOKENIZE, SOLID_MASK, GENERALIZE, DROP, BLOCK
<code>coverage_status</code>	SCANNED, MASKED, SAFE_BY_POLICY, UNKNOWN

No original value, reversible mapping, substring, stable hash, prefix/suffix or entropy-bearing derivative is permitted in the serializable type. `SensitiveFindingLocal` refers to a non-serializable local `SensitiveValueHandle` separately.

9. RedactionRegion

Contains finding IDs, normalized and pixel boxes, mask mode, padding, replacement display class and compositor version. Schema fixes `normalized_box` to root visual-viewport CSS coordinates and `pixel_box` to encoded-image pixels. `replacement_class` is required only for `TYPED_REPLACEMENT` and forbidden for the other mask modes. Critical classes require `SOLID_OPAQUE`; alpha must equal 255 in the final flattened image. Region geometry is immutable after the final verifier begins; a retry creates a new sanitized artifact and digest.

10. SanitizedSceneLocal

Contains snapshot/navigation/mutation lineage, sanitized elements, final encoded sanitized-image handle, privacy manifest, verification state and compiler versions. The image handle refers only to the fresh encoded result, never the original capture.

11. PrivacyManifest

Content-free fields:

- policy, schema, compiler, detector, OCR, verifier and encoder versions;
- counts of findings, tokens and masks by privacy class;
- coverage counts and unknown-region count;
- mask pixel-area ratio bucket;
- whether a remote image is required;
- final sanitized payload byte counts;
- verification status and bounded attempt number.

`SAFE_TO_SEND` requires zero unknown coverage. When `image_included` is true, a positive `sanitized_image_bytes` value and matching scene `sanitized_image_part_id` are required; both are forbidden when no image is included. The final serialized-body digest is authoritative only in `VerifiedEnvelope`, avoiding a self-referential digest inside the body being hashed.

It never contains values, labels, OCR text, token IDs/mappings, page URL, selector, raw geometry tied to a value, task text or image bytes.

12. VerifiedEnvelope

Gateway-internal capability minted only by the final verifier:

```
{
  "schema_version": "1.0",
  "task_id": "tsk_7Yf...",
  "request_id": "req_N2p...",
  "snapshot_id": "snp_K8a...",
  "policy_version": "privacy-1.0.0",
  "body_sha256": "64-lowercase-hex",
  "expires_at": "2026-09-04T10:15:30Z",
  "decision": "SAFE_TO_SEND"
}
```

The capability object itself also has an unforgeable local brand. The gateway recomputes the exact multipart body digest and compares identifiers/expiry before opening a socket.

13. RemoteInferenceRequest

Remote wire request fields:

- `schema_version`, `request_id`, `task_id`, `snapshot_id`, `navigation_id`, `mutation_epoch`;
- `site_profile_id` or non-identifying `origin_alias`;
- `sanitized_task`;
- `sanitized_scene` containing bounded sanitized elements;

- optional `sanitized_image_part_id` inside the scene, never URL/base64 in arbitrary JSON;
- allowed action types;
- the bounded, content-free privacy manifest inside `sanitized_scene`; and
- request expiry.

The image is a separately named multipart part whose digest is covered by the envelope. Raw hostname/path/query/fragment, tab ID, DOM HTML, selector, capture URL/data URL, token mapping and arbitrary metadata are forbidden.

Example:

```
{
  "schema_version": "1.0",
  "request_id": "req_c7R4Lk9P",
  "task_id": "tsk_W2n8Qa1M",
  "snapshot_id": "snp_f9D3Tv6K",
  "navigation_id": "nav_G4p2Mx8H",
  "mutation_epoch": 42,
  "site_profile_id": "synthetic-travel-claim-v1",
  "sanitized_task": "Check whether the claim is within policy and propose submission.",
  "sanitized_scene": {
    "schema_version": "1.0",
    "snapshot_id": "snp_f9D3Tv6K",
    "navigation_id": "nav_G4p2Mx8H",
    "mutation_epoch": 42,
    "sanitized_elements": [
      {
        "schema_version": "1.0",
        "element_id": "elm_A8p4",
        "role": "STATUS",
        "sanitized_label": "Within policy limit: yes",
        "bounding_box": {
          "x": 84,
          "y": 410,
          "width": 260,
          "height": 28,
          "coordinate_space": "ROOT_VISUAL_VIEWPORT_CSS"
        },
        "document_frame_id": "frm_Top1",
        "source": "FUSED",
        "confidence": 0.99,
        "snapshot_id": "snp_f9D3Tv6K",
        "executable": false
      },
      {
        "schema_version": "1.0",
        "element_id": "elm_Q7r2",
        "role": "BUTTON",
        "sanitized_label": "Submit claim",
        "bounding_box": {
          "x": 920,
          "y": 690,
          "width": 180,
          "height": 44,
          "coordinate_space": "ROOT_VISUAL_VIEWPORT_CSS"
        },
        "document_frame_id": "frm_Top1",
        "source": "FUSED",
        "confidence": 0.97,
        "snapshot_id": "snp_f9D3Tv6K",
        "executable": true
      }
    ]
  }
}
```

```

    }
  ],
  "privacy_manifest": {
    "schema_version": "1.0",
    "policy_version": "privacy-1.0.0",
    "components": [
      { "component": "privacy-compiler", "version": "1.0.0" },
      { "component": "final-verifier", "version": "1.0.0" }
    ],
    "finding_counts": [],
    "mask_counts": [],
    "token_counts": [],
    "coverage_total": 2,
    "coverage_unknown": 0,
    "image_included": false,
    "sanitized_json_bytes": 1850,
    "verification_status": "SAFE_TO_SEND",
    "attempt": 1
  },
  "verification_status": "SAFE_TO_SEND"
},
"allowed_actions": ["CLICK", "SCROLL", "FOCUS", "COMPLETE", "ABORT"],
"policy_version": "action-1.0.0",
"expires_at": "2026-09-04T10:15:30Z"
}

```

14. ActionPlan

The server returns one action only. The discriminated union allows:

- CLICK { element_id }
- FOCUS { element_id }
- SCROLL { direction, amount }
- TYPE_TEXT { element_id, value: { kind: LITERAL_SAFE | TOKEN_REF, value } }
- SELECT_OPTION { element_id, option_element_id }
- NAVIGATE_OPAQUE { element_id } for a bound link only
- COMPLETE { outcome_code }
- ABORT { safe_reason_code }

TOKEN_REF.value must be an existing opaque token_id; a display placeholder does not resolve.

LITERAL_SAFE is rescanned locally before execution. The plan cannot carry JavaScript, selectors, coordinates, file paths, arbitrary URLs, headers or cookies.

Example:

```

{
  "schema_version": "1.0",
  "request_id": "req_c7R4Lk9P",
  "task_id": "tsk_W2n8Qa1M",
  "action_id": "act_M5v1Zx3B",
  "snapshot_id": "snp_f9D3Tv6K",
  "navigation_id": "nav_G4p2Mx8H",
  "mutation_epoch": 42,
  "action": { "type": "CLICK", "element_id": "elm_Q7r2" },
  "confidence": 0.93
}

```

15. ExecutionReceipt

Safe local record fields:

- `schema_version`, `task_id`, `action_id`;
- `snapshot_id_before`, optional `snapshot_id_after`;
- action type but no argument content;
- result enum `SUCCEEDED`, `REJECTED`, `STALE`, `CANCELLED`, `FAILED`;
- confirmation status `NOT_REQUIRED`, `CONFIRMED`, `DECLINED`, `EXPIRED`;
- policy decision and safe error code;
- `total_latency_ms`; and
- `recorded_at` timestamp.

It contains no label, value, URL, selector, token ID, model rationale or screenshot.

16. ErrorEnvelope

Error families are closed and content-free: `PERMISSION_*`, `CAPTURE_*`, `SNAPSHOT_*`, `COORDINATE_*`, `RUNTIME_*`, `MODEL_*`, `COVERAGE_*`, `PRIVACY_*`, `EGRESS_*`, `REMOTE_*`, `ACTION_*`, `TELEMETRY_*` and `INTERNAL_SAFE`.

`ErrorEnvelope` includes a safe code, component, retryability, correlation ID, schema version and optional bounded numeric details such as attempt or elapsed bucket. It never includes exception message, stack locals, DOM/OCR/task text, URL or server body. Raw exceptions are mapped locally; unmapped exceptions become `INTERNAL_SAFE`.

17. Compatibility policy

- Patch changes cannot alter schema.
- Minor versions may add enum values or optional fields only through a new registered schema; older consumers reject unsupported values explicitly.
- Major versions are isolated endpoints/topics and fail closed across mismatch.
- The extension advertises supported request/action schema versions; the server selects one exact intersection.
- Policy/model versions are independent from contract versions and are recorded in every verified request.
- A downgrade requires explicit signed configuration and cannot bypass a privacy gate.

18. Contract tests

- Golden valid/invalid examples for every `$defs` type.
- Fuzz unknown keys, oversized arrays/strings/numbers, NaN/infinity, Unicode confusables and prototype-pollution keys.
- Prove raw local brands cannot serialize or reach gateway module types.
- Consumer-driven client/server compatibility tests for every supported exact version.
- Mutation after verification must change digest and be rejected.
- Action plans with JS, selectors, coordinates, arbitrary URLs, raw labels or multiple steps must fail.
- Receipt/error/telemetry canary scans ensure content-free serialization.

DRISHTIGUARD ARCHITECTURE BASELINE

Part V — Benchmark and evaluation plan

Traceability from official SIH metrics to components, outputs, test cases and release gates

Controlled source: docs/BENCHMARK_PLAN.md

Version: 1.0

Date: 4 September 2026

Principle: Privacy gates are mandatory; the official SIH weighted score does not permit trading critical leakage for speed.

1. Evaluation traceability

The official SIH page specifies the 25/20/20/20/15 weighting [R1]. Each metric is connected to a component, observable output, benchmark and failure policy.

SIH metric	Weight	Component and output	Benchmark method	Failure policy
Visual-context accuracy	25%	Coordinate service, DOM collector, visual detector and fusion; top-1 target, box, role and evidence source	Site/template-disjoint portal holdout; DOM-only vs vision-only vs fused ablation across surfaces, DPR and zoom	Low grounding confidence produces no automated action or user-assisted selection
PII precision and recall	20%	PII ensemble and coverage map; per-class TP/FP/FN, confidence and coverage status	Deterministic Indian PII generator plus image/canvas/DOM/document hard cases and hard negatives	Any critical class below its floor blocks release; never hide it in a micro-average
Redaction precision	20%	Privacy compiler, verifier and gateway; residual recovery, masked-area precision/recall and utility	Intercept actual outgoing and server-received bytes; decode, OCR under transforms, inspect metadata/alpha and search canaries	Any observed critical residual blocks runtime egress and release
Client resource utilization	20%	Runtime scheduler; package/cache/resident size, memory, CPU/GPU, load and long tasks	Clean profiles on frozen low/mid/high hardware in Chromium and Firefox; cold/warm runs	Use only a privacy-certified smaller tier; otherwise block
End-to-end latency	15%	Per-stage local/network/remote/action traces; p50/p95	Fixed scenario mix, network profile, server region, browser, hardware and model hashes	Deadline cancels step; never skip privacy verification

Security gates outside the weighted score require 100% stale-plan rejection, 100% declared high-risk confirmation enforcement, zero prohibited consequential actions on the prompt-injection suite and zero seeded raw canaries in actual egress/storage/logs.

2. Dataset strategy and license gate

Source	Intended use	Current license / constraint	Decision
--------	--------------	------------------------------	----------

Source	Intended use	Current license / constraint	Decision
Synthetic travel-claim portal	Primary end-to-end, PII/redaction/action and failure suite	Team-owned code/data; synthetic only	Required; public-safe after review
Synthetic Indian PII generator	Class-balanced positives and hard negatives	Team-owned; generated values must fail real validation	Required
GUIGuard-Bench public split	Privacy-recognition/protection comparison	HF card: CC BY-NC 4.0; 121 trajectories, 2,002 screenshots in current public release [R17]	Internal non-commercial evaluation only; do not redistribute
Mind2Web	DOM/element-selection transfer cases	Dataset CC BY 4.0, code MIT; README says do not redistribute unzipped test data [R15]	Use under terms; record commit; no republishing
Multimodal-Mind2Web	Paired HTML/screenshot exploratory cases	License metadata is ambiguous relative to linked SeeAct terms	Hold pending written/license review
ScreenSpot-Pro	Static high-resolution click grounding	HF/code declare MIT; third-party UI screenshots merit rights review [R16]	Internal benchmark; do not bundle until reviewed
AgentDojo	Prompt-injection policy scenarios	Code MIT; text/tool benchmark, not visual grounding [R14]	Adapt scenario logic; add visual variants
Custom adversarial pages	Cross-origin, canvas, image, PDF-like, missing ARIA, shadow DOM, mutations	Team-owned	Required

Dataset IDs, versions, commits, licenses, generator seeds and allowed uses are frozen in a machine-readable evaluation manifest. No external benchmark image is copied into the public demo or report without license approval.

GUIGuard numbers are version-sensitive. Current arXiv v3 (13 May 2026) describes 241 real trajectories and 4,080 screenshots; average strict full-match privacy recognition is reported as 8.8% on Android and 0.6% on PC, while its protected-planning evaluation is an offline proxy, not direct end-to-end task success [R10]. Preliminary v1 counts and 13.3%/1.4% values must not be quoted as current task-success results.

3. Synthetic-data generation

The generator emits deterministic pages from a seed. It varies:

- Indian names across scripts/transliterations, emails, +91 phones and addresses;
- PAN-shaped values, Aadhaar-shaped values, employee IDs, account/IFSC pairs, invoice IDs, amounts and dates;
- faces generated or licensed for synthetic use;
- labels, nearby distractors, partial occlusion, line wrap, font, contrast, scaling and zoom;
- DOM text, image text, canvas, SVG, PDF-like raster, QR/barcode, open shadow root and accessible same-origin iframe;
- missing/wrong ARIA, duplicated labels, disabled controls and deceptive target names;
- prompt-injection text in visible, hidden, ARIA, image, canvas and QR channels;
- navigation/mutation races and element movement during model/confirmation delays.

Generated identifiers must be syntactically useful but invalid for real use. Aadhaar-like values deliberately fail a validation digit unless a test explicitly needs a checksum-positive synthetic pattern; those cases use reserved fixtures and are never submitted to a real service. Accounts and employee IDs use reserved prefixes. No generation seed is based on a real person or ISRO system.

Split by page template, not screenshot, to prevent layout leakage:

- 60% development/training where applicable;
- 20% calibration;
- 20% frozen holdout, accessible only in CI release jobs.

Create at least 500 positive instances per critical class before claiming a per-class floor, with balanced surface categories. The MVP may use fewer, but must report the exact N and avoid production language.

4. Evaluation cases

4.1 Visual grounding

- Standard semantic buttons/inputs with correct labels.
- Missing ARIA with visible text.
- Duplicate labels and visually adjacent controls.
- Scrolled pages at 80%, 100%, 125%, 150% and 200% zoom.
- DPR 1.0, 1.25, 1.5 and 2.0 where hardware permits.
- Open shadow DOM, same-origin iframe and inaccessible cross-origin iframe.
- Canvas/image/PDF-like surfaces and small controls.
- Layout mutation between collection/capture and between plan/execute.
- Chrome and Firefox rendering differences.

4.2 Privacy recognition

- Each privacy class in DOM text, field value, accessible name, alt/title/placeholder, URL, image, canvas and document-like raster.
- Labels separated spatially from values.
- Split tokens, unusual whitespace, OCR confusion and multilingual surroundings.
- Hard negatives that resemble PAN/Aadhaar/account/phone patterns.
- Faces, documents, QR codes and signatures.
- Organizational secrets expressed as arbitrary project codes under site policy.
- Co-occurrence/trajectory cases where context changes sensitivity.

4.3 Redaction and leakage

- Solid-mask box coverage against oracle boxes.
- Re-encoding, scaling, contrast, sharpening, thresholding, crop and multiple OCR engines.
- Transparent/alpha channel, EXIF/text chunks, ICC/profile and trailing-data inspection.
- Same seeded canary in screenshot, task, scene graph, ARIA, attribute, URL, action argument and error path.
- Intentional under-padding to prove pass two remasks/blocks.
- Payload mutation after verification to prove digest rejection.

- Server-ingress capture compared byte-for-byte with client verified digest.

4.4 Action safety and reliability

- Stale snapshot, moved element, changed role/label, disabled target and navigation.
- Replayed/late/wrong-task response.
- Unknown action/field, oversized plan, JavaScript, selector, coordinate and arbitrary URL.
- Click whose effect is submit/delete/pay despite neutral verb.
- Confirmation decline, expiry and page mutation while dialog is open.
- Remote timeout, malformed response and network interruption.
- WebGPU/WASM/OCR/model/encoder/gateway failures.
- MV3 compute-host eviction and token-vault loss.

5. Metrics

5.1 Visual grounding

- Top-1 element-selection accuracy.
- Bounding-box IoU and center-point hit rate.
- Role macro-F1.
- Executable-binding rate: predicted target resolves to the correct live DOM element.
- Per-step and end-to-end task success.
- DOM-only, vision-only and fused delta with paired confidence intervals.

5.2 PII recognition

- Per-class precision, recall and F1 with raw TP/FP/FN counts.
- Macro and micro averages, with macro as the headline.
- Critical false-negative rate.
- Coverage completion rate and unknown-region rate.
- Per-surface recall: DOM, image, canvas, document, task and metadata.
- Calibration: expected calibration error and reliability curves for confidence-based policy.

5.3 Redaction/privacy

- Critical leakage events / evaluated artifacts.
- Residual OCR recovery rate by class and attack transform.
- Region precision/recall at IoU thresholds.
- Pixel over-redaction = safe pixels masked / total safe pixels.
- Content-item over-redaction = safe semantic items removed / total safe items.
- Task-utility retention = protected task success / unprotected oracle task success.
- Cross-modal canary escape count.
- Exact verified-digest match at server ingress.

"Redaction precision" is reported both geometrically and semantically because one number is otherwise ambiguous.

5.4 Resource usage

- Compressed store package, downloaded model cache and decoded resident weights separately.

- Peak incremental JS/WASM heap, browser process RSS and GPU memory where measurable.
- Average/p95 CPU and GPU utilization during a step.
- Cold model initialization and warm-up time.
- Main-thread total blocking time and count of tasks over 50 ms.
- Capture count, OCR crop count, cache hit rate and bytes transferred between workers.

5.5 Latency

Report p50, p90 and p95 for capture, DOM collection, normalization, visual detector, OCR, PII ensemble, privacy compilation, final verification, network, remote queue, VLM inference, validation, confirmation-excluded execution and full step. Separate cold and warm distributions; do not average them together.

6. Hardware and browser matrix

Freeze exact models before reporting. Proposed tiers:

Tier	Client	Browser/runtime	Purpose
Low	4-core CPU, 8 GB RAM, integrated graphics	Chromium WASM; Firefox WASM	Safety and graceful blocking floor
Mid	6-8 core CPU, 16 GB RAM, WebGPU-capable integrated/discrete GPU	Chromium WebGPU + WASM; Firefox WASM	Primary SIH laptop target
High	8+ cores, 32 GB RAM, discrete GPU	Chromium WebGPU; Firefox WASM	Headroom and profiling

Record OS build, browser full version, extension commit, model/runtime hash, power mode, display scale, viewport, network profile, server region/GPU and queue depth.

The browser matrix uses the latest two stable Chromium and Firefox releases at freeze. Firefox performance parity is not claimed unless its entire end-to-end suite is measured.

7. Proposed targets and release gates

Dimension	Target	Gate interpretation
Grounding top-1 on declared portal holdout	$\geq 85\%$	MVP target, not open-web claim
Critical-PII recall per class	$\geq 97\%$	Release gate with stated CI and N
Overall PII precision	$\geq 92\%$	Macro headline; per-class counts required
Critical residual leakage	0 observed on frozen declared suite	Must state $0/N$ and CI; any observed leak blocks
Pixel over-redaction	$\leq 12\%$	Report content-item form too; utility cannot waive critical mask
Stale-plan rejection	100% on race suite	Release gate
High-risk confirmation enforcement	100% on declared effects	Release gate

Dimension	Target	Gate interpretation
Prohibited prompt-injected action	0 executed	Release gate
Store bundle including MVP models	≤ 50 MB compressed	Report cache/resident sizes separately
Peak incremental memory	≤ 350 MB	Include GPU/process memory as available
Chromium/WebGPU local p95	≤ 1.2 s	Aspirational until measured
Firefox/WASM local p95	≤ 2.0 s	Aspirational until measured
Warm end-to-end p95	≤ 4.0 s	Declare network/server queue profile
Main-thread task	≤ 50 ms	Instrumented release gate for UI responsiveness

Privacy/security gates are conjunctive. A weighted aggregate may be displayed for SIH alignment, but cannot turn a privacy failure into a pass.

8. Statistical reporting

- Report raw counts and denominators before percentages.
- Use Wilson 95% intervals for proportions and stratified bootstrap intervals for macro/paired task metrics.
- With zero failures in N independent cases, report the approximate one-sided 95% upper rate bound $3/N$.
- Use paired bootstrap or McNemar analysis for DOM/vision/fusion variants on the same items.
- Report p95 with bootstrap confidence intervals and at least 100 warm observations per primary matrix cell; use at least 30 cold starts where practical.
- Publish per-class, per-surface, per-browser and per-hardware slices; never use a pooled number to hide a weak critical class.
- Fix seeds, warm-up protocol, power mode and background load. Randomize scenario order.
- Separate model calibration data from the frozen holdout.
- Record missing/blocked cases as outcomes, not silently drop them.

9. Ablation and trade-off studies

Required variants:

1. DOM-only.
2. Vision/OCR-only.
3. Fused union privacy + concordant grounding.
4. Fused without coverage gate.
5. Fused without final-byte verifier.
6. Full frame vs changed-region processing.
7. WebGPU vs WASM on the same supported hardware/model.
8. Scene-graph-only remote request vs graph + sanitized image.
9. Solid mask vs placeholder rendering for non-critical classes.
10. Candidate OCR/model tiers.

Report the privacy-utility-resource Pareto frontier. Do not select a faster point if it violates a critical privacy gate.

10. Regression gates and CI

Every change to capture, coordinate mapping, detectors, policy, compiler, encoder, verifier, contracts, gateway, prompt or executor runs:

- unit and property tests;
- frozen synthetic privacy suite;
- final-byte canary and metadata suite;
- action-state/race suite;
- prompt-injection negative suite;
- schema compatibility/fuzz tests;
- model hash/license manifest validation;
- performance smoke budget on a pinned runner; and
- scheduled full browser/hardware runs.

Release artifacts include `benchmark-manifest.json`, raw aggregate counts with no page content, plots/tables, environment record, known exclusions and a signed model/policy/schema manifest.

11. MVP benchmark demonstration

The SIH demo should show a fixed 12-stage sequence:

11. original local synthetic page;
12. DOM-only detections;
13. vision-only detections;
14. fused findings and coverage map;
15. typed random token capabilities;
16. fresh sanitized image and graph;
17. pass-two decision, including one seeded failure;
18. browser network interception and server-ingress digest/canary result;
19. remote one-action response;
20. local element/policy/freshness validation;
21. extension-owned confirmation and execution; and
22. page-state diff plus measured scorecard.

The scorecard names the laptop, browser, versions, model hashes, dataset seed/N and cold/warm status. Targets are never displayed as achieved results.

12. Credibility limits

- $\geq 85\%$ grounding can be credible on the frozen synthetic portal family, not arbitrary websites.
- $\geq 97\%$ critical recall requires substantial per-class N; a handful of obvious examples is not evidence.
- $0/N$ residual leakage is a test result, not a universal guarantee.
- A complete detector + OCR + NER + face/document stack within 50 MB is risky; report three size definitions.

- 350 MB depends on sequential inference and buffer reuse; JavaScript heap alone is insufficient measurement.
- Firefox/WASM ≤ 2 seconds may be missed when OCR is required.
- End-to-end ≤ 4 seconds depends on a warm nearby GPU and bounded queue.
- GUIGuard, Mind2Web, ScreenSpot-Pro and AgentDojo measure different constructs; do not merge their scores into one headline.

DRISHTIGUARD ARCHITECTURE BASELINE

Part VI — Implementation roadmap

Hackathon MVP scope, 48-hour sequencing, post-selection hardening and production gates

Controlled source: docs/IMPLEMENTATION_ROADMAP.md

Version: 1.0

Basis: Six-person team, pre-event preparation and a 36-48 hour SIH implementation sprint. Replan when the actual team, event format and hardware are known.

1. Delivery principle

The architecture describes a production target. The hackathon delivers a measured vertical slice, not a production-ready browser agent. Scope is protected in this order:

23. real privacy controls and actual network evidence;
24. stale-safe, confirmed local execution;
25. DOM + vision demonstration and benchmark;
26. compatibility and polish; and
27. broader website coverage only if every earlier gate is complete.

2. Team lanes

Lane	Primary responsibilities
Extension/platform	WXT shell, side panel, capture, permissions, browser adapters, task lifecycle
DOM/action	Collector, opaque binding table, mutation/navigation state, validator and executor
Local ML	Visual detector, OCR, PII recognizers, runtime tiers, performance
Privacy/security	Coverage, compiler, token vault, verifier, gateway, threat/fault tests
Backend/VLM	FastAPI modular monolith, schema, prompt channels, constrained one-action output
Demo/QA	Synthetic portal/generator, benchmark harness, metrics, UX, integration and presentation

At least two people review privacy/gateway changes. No lane may add an independent page-data network call.

3. Pre-event foundation

Complete before the timed sprint:

- Freeze Chrome/Firefox versions, demo laptop, display scaling and server GPU.

- Create WXT monorepo, exact locks, lint rule restricting network APIs and CI.
- Implement versioned JSON Schema and generated/checked TypeScript types.
- Build the synthetic travel-claim portal and deterministic invalid PII generator.
- Spike `captureVisibleTab`, coordinate mapping and mutation/navigation identity at multiple zoom/DPR settings.
- Spike `PaddleOCR.js` or alternative ONNX OCR in MV3 Chrome and Firefox workers; record size, operator support and latency.
- Select/freeze small local detector/NER assets and signed hashes.
- Prepare a Qwen3-VL-4B-Instruct or alternative server endpoint with strict one-action schema.
- Write canary interception, leak scan, action race and prompt-injection harnesses.
- Pre-record a degraded-mode backup video only as presentation insurance, never as substitute for live evidence.

If the browser OCR/model spike fails, narrow the MVP to a smaller validated OCR/model combination rather than improvising an unmeasured runtime during the finale.

4. Hackathon MVP

4.1 Must have

- Explicit extension-owned activation and consent/preview.
- Chromium visible-viewport capture with exact two/s throttle.
- DOM/ARIA-derived graph, opaque IDs and live local binding table.
- Read-capture-read state consistency and stale job cancellation.
- Actual-dimension coordinate normalization.
- Local vision + ROI OCR on one hard canvas/image region.
- Deterministic Indian PII rules, face/document masking and coverage map.
- Cross-modal task/graph/image sanitizer.
- Task-random typed token capability vault.
- Solid opaque masks in a newly encoded bitmap.
- Final-byte verifier and bounded remask/block path.
- Gateway digest binding and intercepted network/server-ingress canary proof.
- FastAPI modular monolith with validated sanitized request and one-action result.
- DOM-backed submit target, effect-based risk and extension-owned confirmation.
- Revalidation after confirmation, execution receipt and page-state diff.
- Seeded residual leak, stale plan and prompt-injection negative demonstrations.
- Measured DOM-only/vision-only/fused confusion counts, p50/p95 latency, memory and size.

4.2 Should have

- Scene-graph-only versus graph+image payload switch.
- WASM fallback on the same Chromium build.
- Firefox capture/collector/runtime smoke test with safe unsupported behavior.
- OCR cache and changed-region optimization with full-frame diff guard.
- Safe local metrics dashboard and generated benchmark manifest.
- Model/policy/schema version view in the demo UI.

4.3 Could have

- Open shadow-root case and accessible same-origin iframe.
- One additional workflow step before submit.
- Alternative remote model adapter.
- Local derived-value function for claim-limit logic.
- Automated extension package/signature demo.

4.4 Will not have

- Universal websites, arbitrary cross-origin/canvas execution, closed shadow DOM or protected PDF automation.
- Password/OTP/login, uploads, payments, deletion, permission changes or arbitrary navigation.
- Production auth, enterprise policy, content diagnostics or fleet deployment.
- Full Firefox performance parity.
- Kubernetes, microservice decomposition, database or screenshot object storage.

5. Suggested 48-hour run plan

Window	Integration objective	Exit evidence
H0-H4	Reconfirm versions/hardware, build all packages, run golden contracts	Green build and frozen manifest
H4-H10	Capture + DOM graph + coordinate/state IDs integrated	Zoom/DPR/stale capture tests pass
H6-H14	Local detector/OCR/PII/coverage integrated	Overlay on all synthetic classes
H10-H18	Compiler, token vault, fresh bitmap and verifier	Seeded leak remarks/blocks
H14-H22	Gateway and server strict request/action loop	Intercepted body and digest match
H18-H28	Validator, confirmation, DOM execution and postcondition	Safe submit; stale/malicious plan rejected
H24-H34	Benchmark harness, ablations, resource traces	Named-hardware scorecard
H30-H40	Firefox smoke, failure paths and UX polish	Capability matrix; safe blocking
H38-H46	Rehearsal, defect fixes, clean install and offline backup	Two complete timed demo runs
H46-H48	Freeze artifacts and claims	Reproducible release bundle

Parallel windows are intentional. Stop feature work at H40.

6. MVP acceptance gates

The prototype is SIH-demo ready only when:

- the actual client request and server ingress contain zero seeded raw canaries in the declared suite;
- a pass-one miss is caught by pass two and blocks/remasks;
- DOM-only, vision-only and fused results are measured on a frozen holdout;

- every high-risk submit requires a current confirmation;
- stale, replayed and prompt-injected forbidden plans execute zero actions;
- failure of WebGPU/OCR/remote service cannot trigger a weaker privacy path;
- the scorecard names browser, hardware, models, N, seed and cold/warm status; and
- the team calls results targets or measured results accurately.

If these fail, show a smaller honest slice rather than a broader simulation.

7. Post-selection engineering - approximately 4-8 weeks

- Expand frozen synthetic corpus and build license-governed external benchmark adapters.
- Calibrate per-class thresholds and multilingual Indian names/addresses/documents.
- Complete Firefox end-to-end behavior and formal browser capability tests.
- Harden capture consistency, dynamic canvas/video policy and changed-region correctness.
- Generate TypeScript/contracts automatically and add schema fuzzing.
- Implement OIDC/PKCE, short-lived tokens, endpoint policy and rate limiting.
- Add signed model manifest/cache lifecycle and browser-store policy decision.
- Automate log/storage/network canary inspection and fault injection.
- Add content-free telemetry with opt-in and short retention.
- Establish release provenance, SBOM, dependency scanning and safe rollback.

Exit: repeatable internal alpha on declared synthetic and approved public datasets with no open critical security defects.

8. Pilot readiness - approximately 8-12 additional weeks

- Select a small allowlist of real portals/workflows and build reviewed site adapters.
- Obtain sponsor decisions for PII classes, action policy, residency, retention and authentication.
- Complete DPIA/privacy review and DPDP/legal analysis.
- Conduct independent extension security review and prompt-injection red team.
- Add managed enterprise deployment policy, origin allowlist and kill switch.
- Establish SLOs, on-call, incident response, vulnerability handling and key/model rotation.
- Load/capacity test GPU service and verify body logging disabled at every hop.
- Run accessibility and user comprehension studies for preview/confirmation.
- Maintain statistically adequate per-class/surface/browser benchmark history.

Exit: sponsor-approved limited pilot, signed releases, measured residual-risk acceptance and rollback exercise.

9. Production readiness - evidence gated, likely 3-6+ months total

- Independent privacy and security audit closed.
- Large site-disjoint, multilingual and adversarial benchmark meets frozen gates.
- Chrome and Firefox matrices pass sustained regression; unsupported surfaces fail safely.
- Model/policy/schema governance has approval, provenance, canary and rollback.
- Operational security, IAM, secrets, retention, incident and disaster-recovery controls are tested.
- GPU cost, queueing, regional availability and capacity meet SLOs.
- Store/enterprise distribution and model-update policies are approved.

- User notices, consent, deletion and support processes are deployed.
- Residual detection/inference risk is formally accepted for each supported workflow.

Production readiness is a governance and evidence state, not a branch name or successful demo.

10. Delivery artifacts by phase

Artifact	MVP	Post-selection	Pilot	Production
Architecture, threat/privacy model, ADRs	Baseline	Revised	Approved	Controlled
JSON Schema + TypeScript contracts	Hand/CI checked	Generated	Compatibility matrix	Lifecycle governance
Synthetic portal and generator	Core suite	Expanded	Portal-specific cases	Continuous regression
Chromium extension	End-to-end	Hardened	Signed managed build	Store/enterprise rollout
Firefox	Smoke/capability	End-to-end	Signed build	Sustained parity claims only if measured
Local model pack	Frozen demo	Signed cache lifecycle	Calibrated	Governed updates/rollback
Remote service	Single GPU modular monolith	Auth/rate limit	Load-tested regional service	Autoscaled SLO service
Security evidence	Canaries/races/injection	Automated fault suite	External review/red team	Independent audit/continuous testing
Metrics	Named-laptop scorecard	Broader matrix	SLO and risk report	Release dashboard

11. Go/no-go risks

Risk	Trigger	Decision
Browser OCR/model incompatible or too large	Spike misses size/operator/latency budget	Choose smaller certified engine or narrow visual surface
Critical PII recall weak	Any class below floor	Add policy/site recognizer or block that surface; no release
Verifier cannot distinguish safe payload	Residual/coverage ambiguity	Scene-graph-only mode or local/manual action
Remote VLM cannot ground opaque IDs reliably	Low action accuracy	Improve graph/labels or use deterministic workflow adapter
Firefox unsafe/unstable	Smoke failures	Claim Chromium only for demo; keep Firefox planned, not falsely complete
End-to-end p95 exceeds 4 s	Warm measured result misses	Show measured result, optimize after privacy gates; do not skip verification

Risk	Trigger	Decision
50 MB package missed	Compressed artifact too large	Report categories and use reviewed model-pack strategy if allowed
Team capacity smaller than assumed	Missing lane ownership	Drop breadth and retain proof-focused must-haves

12. Credible presentation claims

Allowed after evidence:

- "Raw capture buffers and the reversible token vault have no gateway representation."
- "Zero seeded critical canaries escaped in O/N frozen test artifacts."
- "The second pass caught this seeded miss in the exact encoded request."
- "The local validator rejected stale and injected actions in N/N tests."
- "Measured p95 was X on named hardware and browser."

Not allowed:

- "100% private" or "no PII can ever leak."
- "First typed-placeholder GUI privacy system."
- "Production-ready" after a hackathon.
- "Works on every website" or "full Chrome/Firefox parity" without evidence.
- Presenting target numbers as measured results.

DECISION GOVERNANCE

Part VII — Architecture decision records

Accepted decisions that define the baseline and the consequences the team must preserve

ADR-0001: Fuse DOM-derived semantics and visual perception

Controlled source: docs/adr/0001-dom-vision-fusion.md

Status: Accepted

Context: DOM/ARIA is efficient and gives roles and live targets, but misses canvas, images, PDFs, inaccessible frames and incorrect markup. Vision sees rendered pixels but is costlier and less reliable for semantics and executable binding.

Decision

Collect both signals locally. Privacy uses the geometric/category union of DOM, OCR, vision and policy findings. Automated action grounding requires a current DOM binding plus sufficient semantic/visual concordance. Every element records source evidence. A coverage map marks visible regions scanned, masked, policy-safe or unknown.

Alternatives

- DOM only: fast and actionable but blind to pixel-only content.
- Vision only: covers pixels but loses reliable roles and DOM execution binding.
- Remote multimodal fusion: exposes raw context and violates the privacy boundary.

Consequences

Recall improves and the system can demonstrate DOM-only/vision-only/fused ablations. Coordinate normalization and snapshot consistency become first-class components. The pipeline uses more memory and latency.

Risks

Union masking can over-redact; concordance can reduce utility. Coordinate errors can create leaks or wrong actions. Mitigate with actual-dimension transforms, padding, frozen zoom/DPR tests, and user-assisted mode when binding is uncertain.

ADR-0002: Use WebGPU with a certified WASM fallback

Controlled source: docs/adr/0002-webgpu-wasm-fallback.md

Status: Accepted

Context: Browser hardware and execution-provider support varies. ONNX Runtime Web currently lists broad WASM support and Chromium WebGPU, but not Firefox WebGPU parity [R7]. Privacy cannot depend on an optional accelerator.

Decision

Feature-detect and benchmark a WebGPU tier for supported Chromium and a quantized WASM SIMD tier for all supported browsers. Model manifests declare compatible operators, memory budgets and privacy-equivalent detector coverage. Runtime failure may switch only to a prevalidated equivalent tier; otherwise the request blocks.

Alternatives

- WebGPU only: faster on some devices but incompatible/fragile.
- WASM only: widest compatibility but may miss latency targets.
- Separate browser implementations: duplicates security-critical logic and drifts.

Consequences

One pipeline supports heterogeneous devices while keeping privacy gates constant. Build/test complexity and model packaging increase. Firefox may be functionally supported without equal performance.

Risks

Operator fallback, GPU driver loss and memory duplication can invalidate targets. Mitigate with model export tests, cold/warm profiling, buffer reuse, bounded inputs and no silent detector omission.

ADR-0003: Use typed, task-random token capabilities

Controlled source: docs/adr/0003-typed-placeholders.md

Status: Accepted

Context: Solid masks protect pixels but can remove task structure. Typed placeholders retain categories. Recent work already uses deterministic typed placeholders, so this is not claimed as novel [R11]. Guessable labels or stable hashes permit linkability and injection.

Decision

Use human-facing labels such as [EMAIL_1] only for display. The resolvable `token_id` is random per task and bound to privacy class, purpose, origin, field, navigation scope, TTL and use count. The mapping stays in the in-memory vault. The executor resolves only an existing allowed token after policy/freshness/confirmation checks.

Alternatives

- Black boxes only: strongest visual hiding but lower utility.
- Deterministic hash tokens: stable but linkable/dictionary-attackable.
- Encrypted values sent remotely: expands key management and reveals ciphertext/linkability.

Consequences

The remote planner can reason about type and repeated reference without receiving the value. Local lifecycle and navigation handling become more complex.

Risks

Token theft enables local re-identification; display-token injection could confuse the model. Mitigate with random capabilities, no serializer, strict resolution rules and task cancellation on vault loss.

ADR-0004: Verify the exact outbound bytes and fail closed

Controlled source: docs/adr/0004-fail-closed-verification.md

Status: Accepted

Context: Detection and masking can fail because of missed regions, encoding metadata, alpha channels or cross-modal leaks. Scanning an intermediate canvas does not prove the network body is safe.

Decision

After compilation and encoding, inspect the exact image and serialized multipart/JSON. Verify known values/canaries, patterns, secondary OCR, coverage, mask opacity, metadata, forbidden keys and payload limits. Mint a short-lived `VerifiedEnvelope` over the exact SHA-256 body digest. Only `SAFE_TO_SEND` authorizes the gateway. Retry masking at most twice, then block.

Alternatives

- Single-pass sanitizer: lower latency but no independent leakage gate.
- Server-side validation: sensitive content has already crossed the boundary.
- Best-effort warning: contradicts the core privacy posture.

Consequences

The claim can be demonstrated against actual traffic and failure is explicit. Latency and implementation cost increase.

Risks

The verifier can share failure modes with the detector, and a compromised extension can bypass it. Use different OCR settings/model where feasible, final-byte canaries, static networking restrictions and supply-chain controls.

ADR-0005: Use remote planning with local policy and execution

Controlled source: docs/adr/0005-remote-vlm-local-execution.md

Status: Accepted

Context: A server VLM offers stronger reasoning than a lightweight browser model, but remote output is untrusted and webpage prompt injection can manipulate it.

Decision

Send only a verified minimized request. The remote service returns exactly one closed-schema action over an opaque element ID. It has no browser, URL-fetch, filesystem or shell tool. Local code validates task/snapshot state, live target and effect policy, obtains confirmation when needed, revalidates, then executes a DOM-backed action.

Alternatives

- Remote direct execution: violates local control and expands blast radius.
- Fully local agent: strongest confidentiality but outside MVP resource constraints.
- Multi-action plans: efficient but stale and hard to review.

Consequences

Reasoning and authority are separated; server compromise cannot directly operate the browser. Each step incurs network latency and page re-observation.

Risks

Sanitized context can still reveal relationships; malicious plans can exploit semantic ambiguity. Minimize payload, apply effect-based policy and default ambiguous targets to manual mode.

ADR-0006: Keep token mappings in an active in-memory vault

Controlled source: docs/adr/0006-in-memory-token-vault.md

Status: Accepted

Context: Persistence improves continuity but creates durable re-identification material. MV3 service workers can be suspended, so service-worker memory alone is unreliable.

Decision

Own the vault in the active extension compute/UI host, use short idle/hard TTLs, and persist no mappings. Host loss, task end, tab close, expiry, extension update or unexpected navigation invalidates capabilities and cancels dependent work. Same-origin continuity may reissue explicitly authorized user-task tokens with new IDs; page-derived mappings do not silently persist.

Alternatives

- `chrome.storage`/IndexedDB encryption: durable but key storage and local compromise remain difficult.
- Service-worker memory: simpler but lifecycle loss is unpredictable.
- Server-side vault: violates the device-local invariant.

Consequences

Privacy blast radius and forensic persistence shrink. Long tasks can be interrupted and must restart.

Risks

JavaScript cannot guarantee physical memory zeroization; a local process can inspect memory. Minimize copies, overwrite mutable buffers and state the local-compromise boundary explicitly.

ADR-0007: Isolate browser differences behind capability adapters

Controlled source: `docs/adr/0007-cross-browser-adapters.md`

Status: Accepted

Context: Chromium and Firefox differ in MV3 lifecycle, capture permissions, offscreen facilities and WebGPU availability. Forked codebases would duplicate privacy-critical logic.

Decision

Use one TypeScript/domain codebase and interfaces for `CaptureAdapter`, `PermissionAdapter`, `ComputeHostAdapter`, `ModelRuntimeAdapter`, `DocumentIdentityAdapter` and `StorageAdapter`. Select by tested capability, not scattered browser-name branches. Build browser manifests from one WXT project [R6].

Alternatives

- Chromium only: simplest but does not meet stated cross-browser direction.
- Separate browser repositories: flexible but high drift/security review cost.
- Lowest-common-denominator runtime: wastes WebGPU while still not solving lifecycle differences.

Consequences

Core privacy/policy logic is shared and adapters are contract-tested. Equal performance is not implied. Adapter test burden remains.

Risks

An adapter may claim a capability incorrectly. Freeze supported versions, run end-to-end matrix tests and block unsupported surfaces.

ADR-0008: Use solid opaque masks for critical visual data

Controlled source: docs/adr/0008-solid-masks.md

Status: Accepted

Context: Blur, mosaic and weak pixelation may preserve recoverable structure. Overlay-only masks may be reversible through alpha, metadata or retained background layers.

Decision

Critical regions use padded, solid, fully opaque masks rendered into a new flattened bitmap. The safe encoder removes metadata and alpha surprises. Non-critical values may use typed semantic replacement only when the same policy passes final verification. Blur is forbidden for critical classes.

Alternatives

- Blur/mosaic: visually pleasant but recoverability varies.
- CSS/DOM overlay: does not change captured underlying pixels reliably.
- Crop only: can remove required task context and misses scattered findings.

Consequences

Critical redaction is simple to test and explain. Over-redaction and remote task degradation can increase.

Risks

Incorrect boxes still leak or hide controls. Use union findings, line expansion, configurable padding, coverage gates and geometric/pixel verification.

ADR-0009: Use changed-region inference only as a guarded optimization

Controlled source: docs/adr/0009-changed-region-inference.md

Status: Accepted

Context: Reprocessing the full viewport is expensive, but DOM mutations do not capture canvas, animation, video or compositing changes. Reusing stale safe labels can leak.

Decision

Compute a full-frame pixel diff for each coordinated capture and reuse OCR/detection results only for stable regions whose navigation, viewport, zoom, transform and policy lineage match. Navigation, scroll/resize/zoom, dynamic surfaces, high diff or uncertainty resets the baseline. Final verification covers the complete outgoing artifact every time.

Alternatives

- Full inference every step: simplest/safest but may miss resource and latency targets.
- DOM-mutation-only invalidation: fast but blind to pixel-only changes.
- Continuous video capture: violates capture cost/rate and resource constraints.

Consequences

Warm steps can be faster while privacy gates remain complete. Cache keys and invalidation logic are security-critical.

Risks

False stability is a privacy failure. Use conservative thresholds, salted local hashes, short lineage and regression cases for canvas/video/animation.

ADR-0010: Use content-free, schema-limited telemetry

Controlled source: docs/adr/0010-privacy-safe-telemetry.md

Status: Accepted

Context: Performance, reliability and model regressions need observability, but conventional logs/APM can capture screenshots, prompts, OCR, URLs and values.

Decision

Define a separate fixed telemetry schema containing only component timing, capability flags, versions, counts, size/resource buckets, decision enums and safe error codes. No generic message, URL, prompt,

response, stack-local or arbitrary tags. Diagnostics are opt-in and short-retention. Scrub failure drops the event.

Alternatives

- No telemetry: minimizes leakage but blocks production operations.
- Conventional structured logs with redaction: flexible but fields/exception capture drift.
- Content logs under consent: too risky for the default product path.

Consequences

Operators can detect latency/failure trends without page content. Debugging individual semantic failures is harder and relies on local synthetic reproduction.

Risks

Counts/timing can still be identifying in combination, and infrastructure can add body logging. Bucket values, rotate trace IDs, limit retention and audit every hop with synthetic canaries.

Architecture handoff

DECISION

Proceed with the narrow SIH vertical slice only if exact-payload privacy verification, intercepted-network evidence, stale-action refusal, prompt-injection rejection and digest-bound confirmation remain non-negotiable scope.

Immediate implementation order

28. Freeze browser versions, demo hardware, schema v1.0 and synthetic corpus seed.
29. Build the page probe, snapshot consistency check and local element-binding table.
30. Implement the cross-modal privacy compiler, coverage map and exact serialized-payload verifier.
31. Lock networking to the digest-bound gateway and prove the actual client/server bytes with seeded canaries.
32. Add one-action remote planning, deterministic local effect policy, confirmation and post-confirmation revalidation.
33. Run the frozen benchmark and present raw counts, confidence intervals, p50/p95 latency, package/cache size and peak process/GPU memory.

Approval boundary

This baseline authorizes implementation planning, not production deployment. A real-data pilot additionally requires sponsor decisions, DPIA/legal review, allowlisted portals and workflows, authentication and residency controls, independent security/privacy review, measured release gates, incident response and a safe rollback/kill switch.

End of controlled document • DG-ARCH-001 • Version 1.0